Git

inky4832@daum.net

Git 개요

개요

분산 버전 관리 시스템 중 하나.
2005년 리누스 토르발스에 의해서 만들어짐



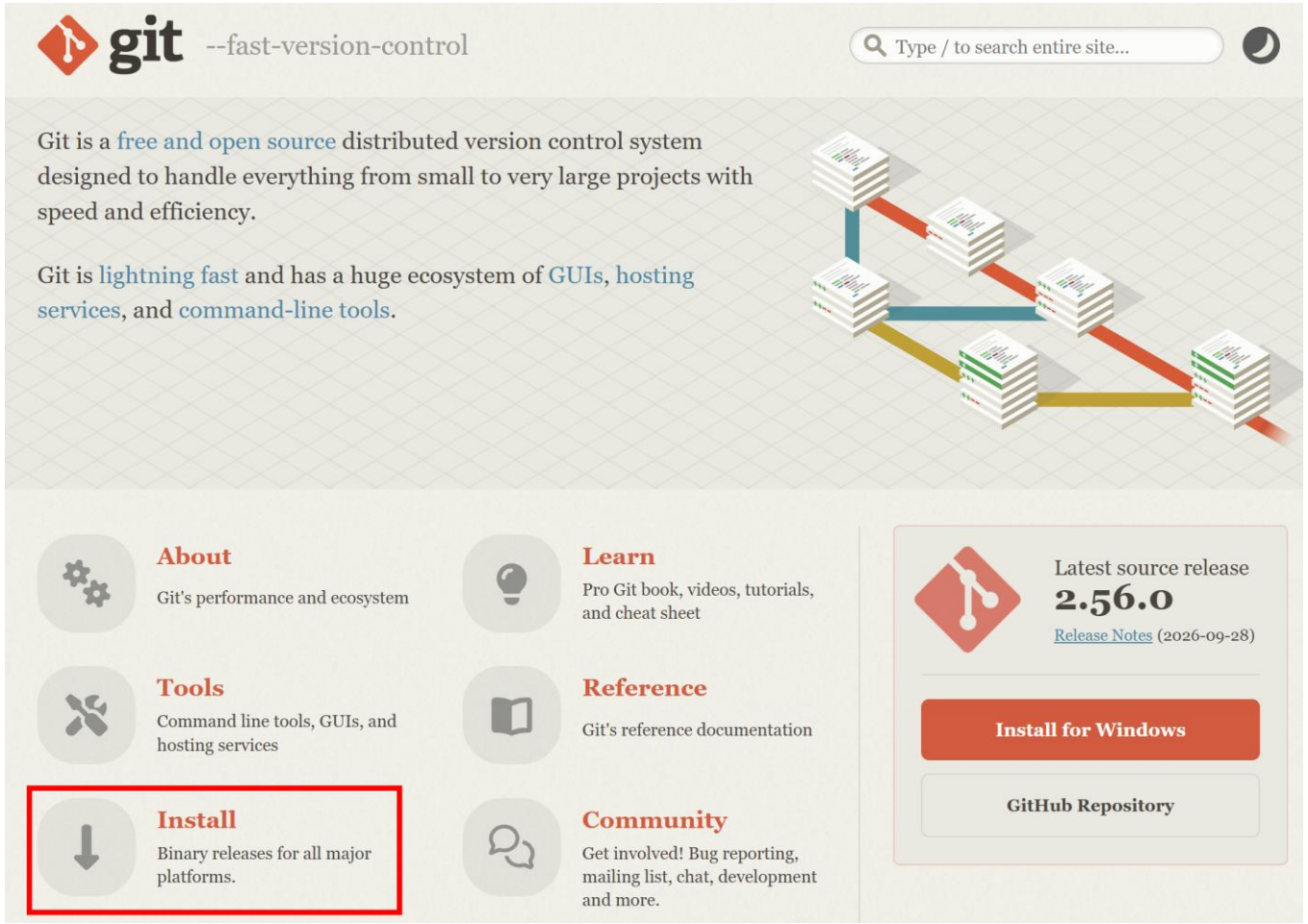
기능

로컬 및 원격 저장소 생성.
로컬 저장소에 파일생성 및 추가.
수정 내역을 로컬 저장소에 제출.
파일 수정내역 추적.
원격 저장소에 제출된 수정 내역을 로컬 저장소에 적용.
main에 영향을 끼치지 않는 브랜치(branch) 생성.
브랜치 병합(merge).
이전 작업과 비교 (diff)
이전 작업으로 복구(reset 및 revert)

2. Git 설치

1) 다운로드

https://git-scm.com/

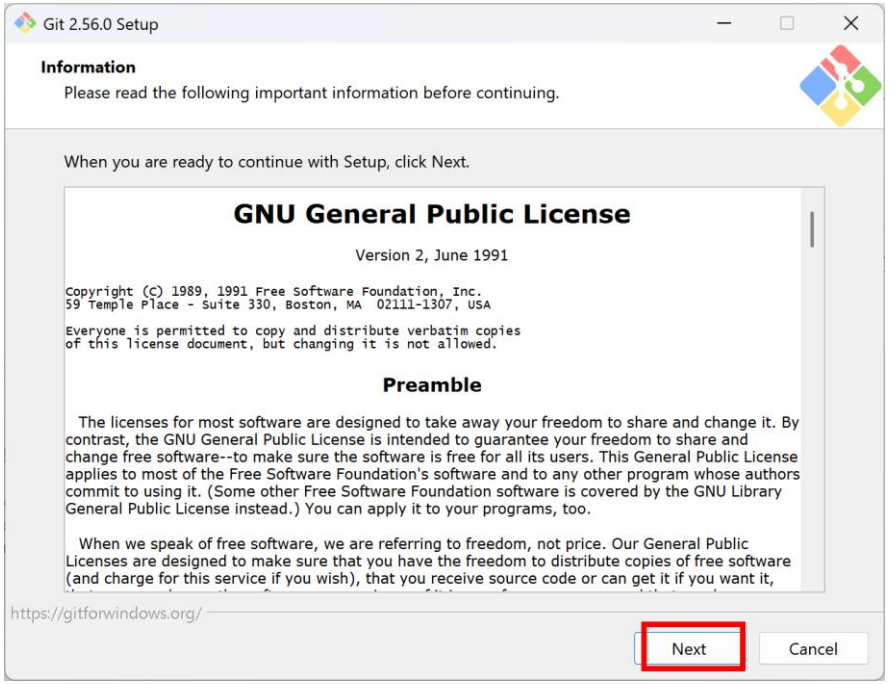


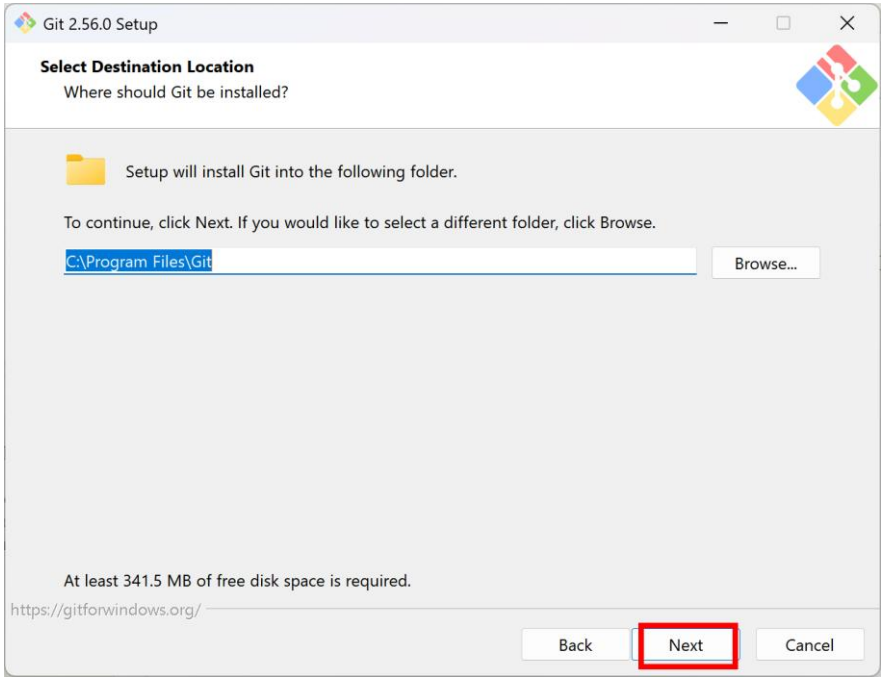
각 운영체제에 맞는 Git 프로그램 다운로드 하기



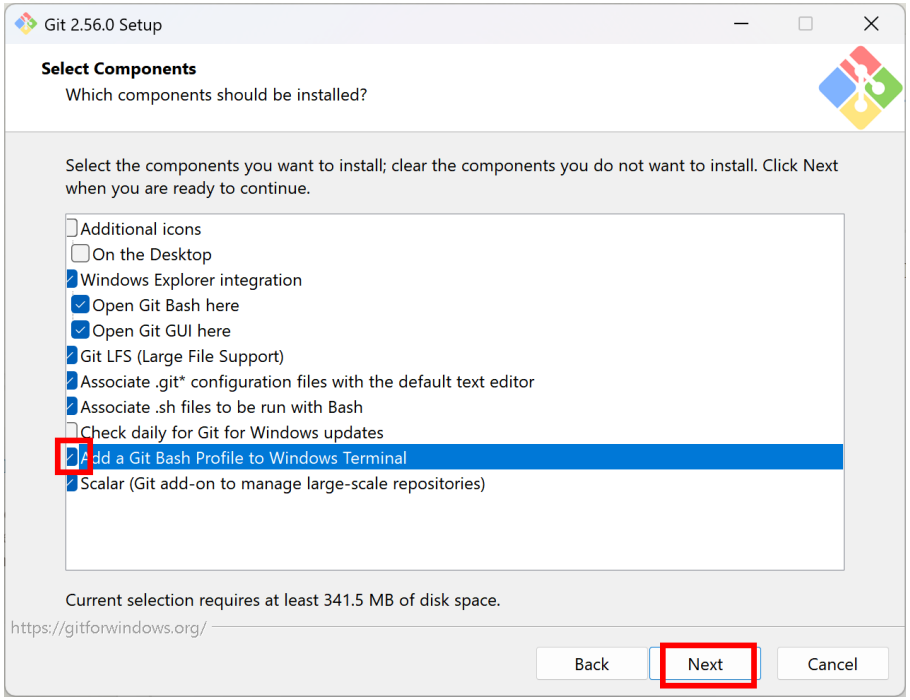
2) 설치

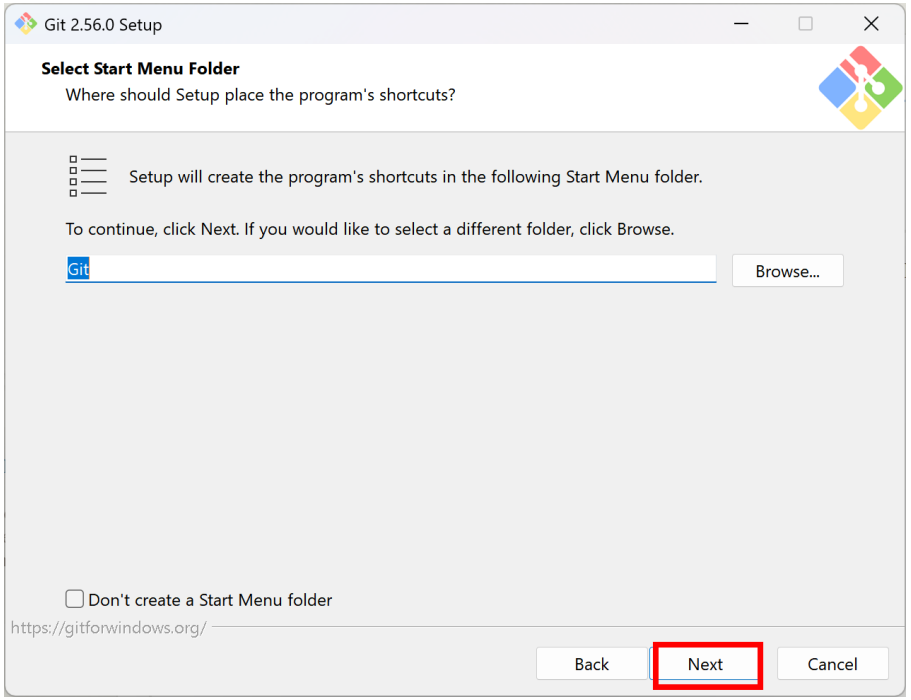
교안을 작성하는 시점의 설치하는 버전은 Git-2.56.0 임.



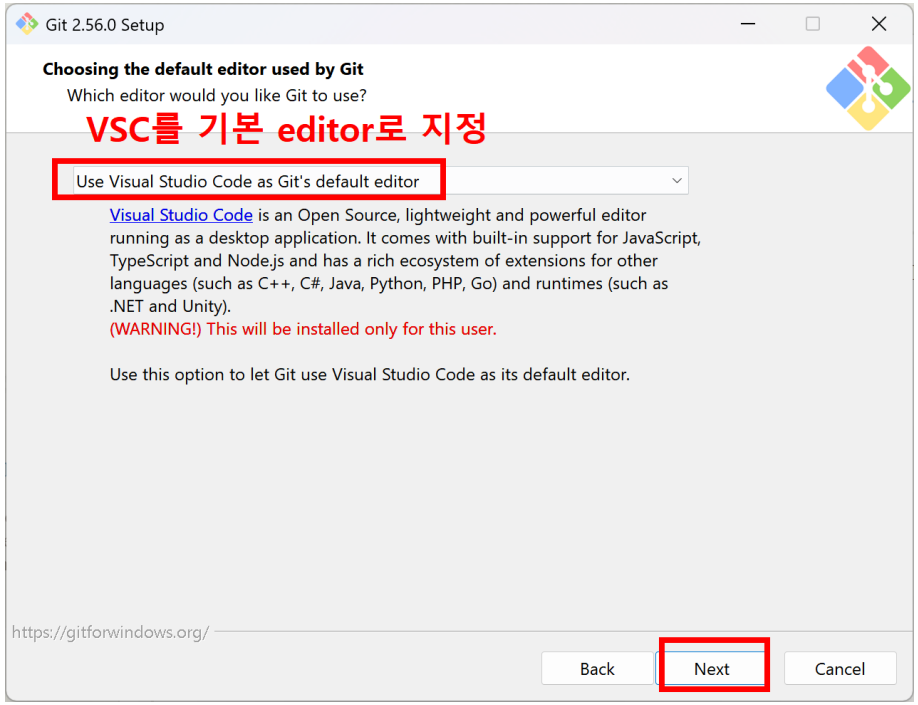


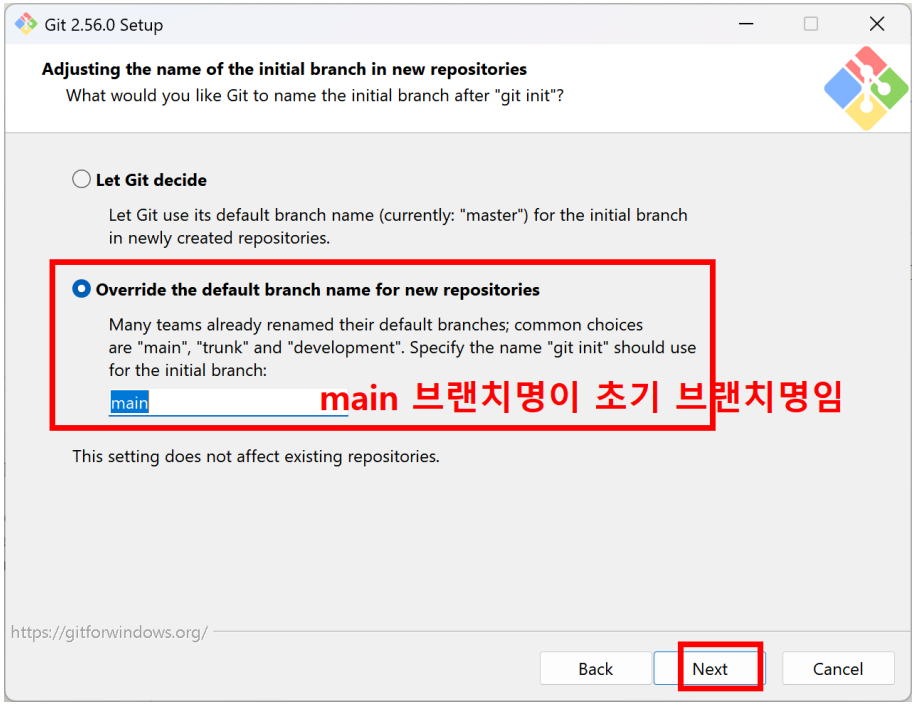
2. Git 설치


Git 2.56.0 Setup
Select Components
Which components should be installed?
Select the components you want to install; clear the components you do not want to install. Click Next when you are ready to continue.
Additional icons
On the Desktop
Windows Explorer integration
Open Git Bash here
Open Git GUI here
Git LFS (Large File Support)
Associate .git* configuration files with the default text editor
Associate .sh files to be run with Bash
Check daily for Git for Windows updates
Add a Git Bash Profile to Windows Terminal
Scalar (Git add-on to manage large-scale repositories)
Current selection requires at least 341.5 MB of disk space.
https://gitforwindows.org/
Back
Next
Cancel

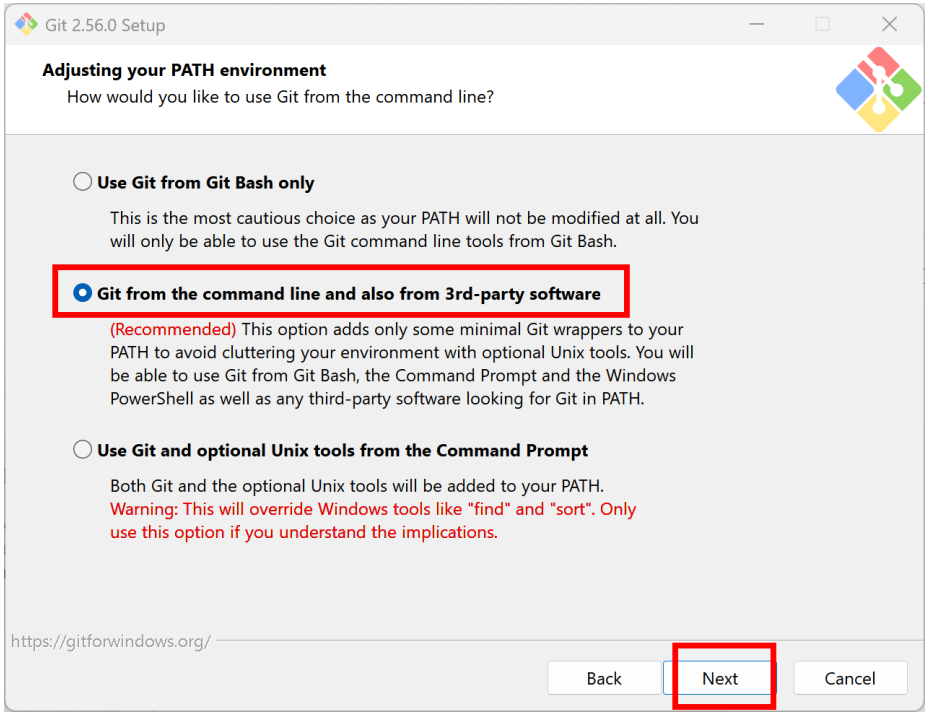

Git 2.56.0 Setup
Select Start Menu Folder
Where should Setup place the program's shortcuts?
Setup will create the program's shortcuts in the following Start Menu folder.
To continue, click Next. If you would like to select a different folder, click Browse.
Git
Browse...
Don't create a Start Menu folder
https://gitforwindows.org/
Back
Next
Cancel

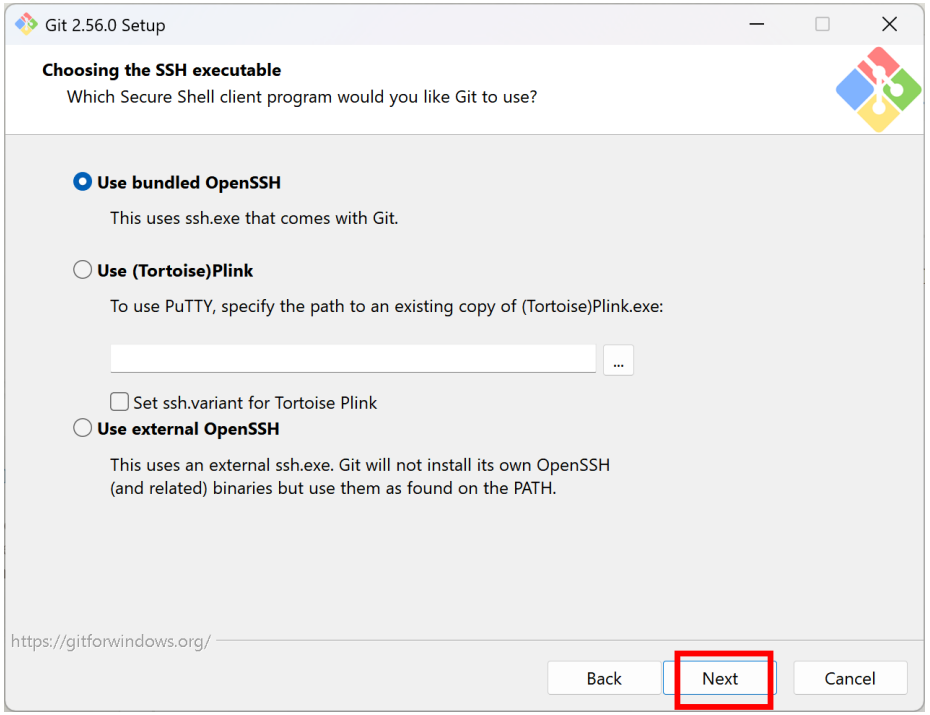
2. Git 설치

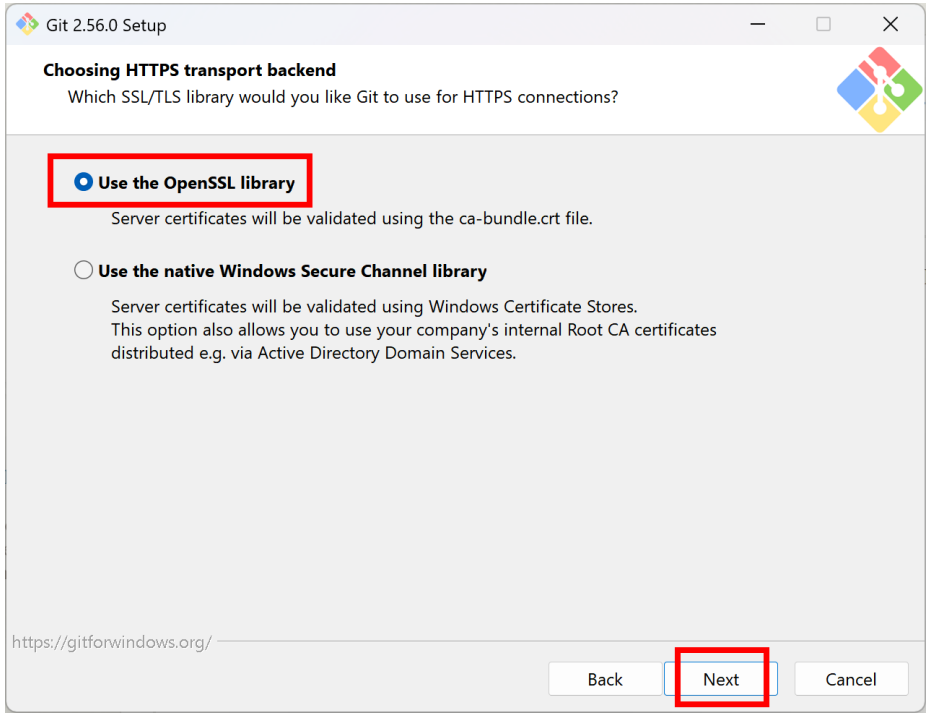

Git 2.56.0 Setup
Choosing the default editor used by Git
Which editor would you like Git to use?
VSC를 기본 editor로 지정
Use Visual Studio Code as Git's default editor
Visual Studio Code is an Open Source, lightweight and powerful editor running as a desktop application. It comes with built-in support for JavaScript, TypeScript and Node.js and has a rich ecosystem of extensions for other languages (such as C++, C#, Java, Python, PHP, Go) and runtimes (such as .NET and Unity).
(WARNING!) This will be installed only for this user.
Use this option to let Git use Visual Studio Code as its default editor.
https://gitforwindows.org/
Back
Next
Cancel

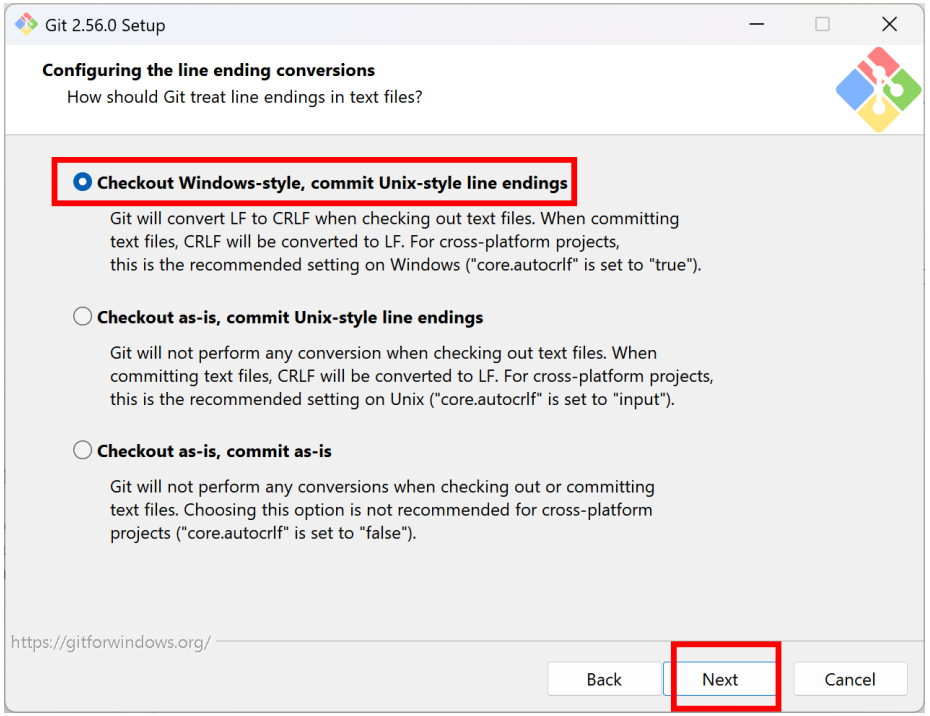

Git 2.56.0 Setup
Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?
Let Git decide
Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories.
Override the default branch name for new repositories
Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:
main
main 브랜치명이 초기 브랜치명임
This setting does not affect existing repositories.
https://gitforwindows.org/
Back
Next
Cancel

2. Git 설치

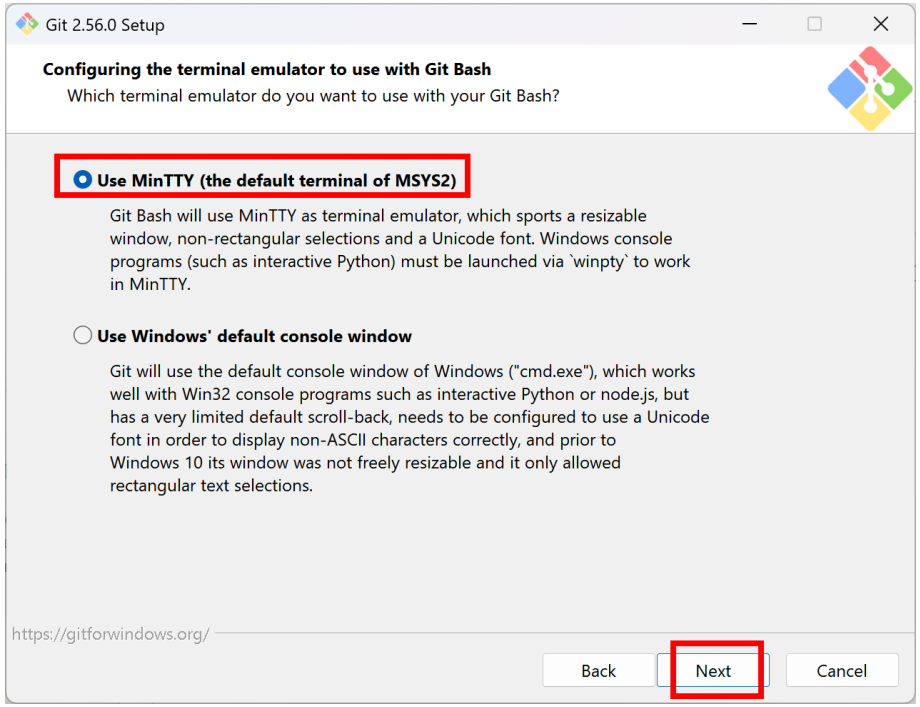

Git 2.56.0 Setup
Adjusting your PATH environment
How would you like to use Git from the command line?
Use Git from Git Bash only
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.
Git from the command line and also from 3rd-party software
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.
Use Git and optional Unix tools from the Command Prompt
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.
https://gitforwindows.org/
Back
Next
Cancel

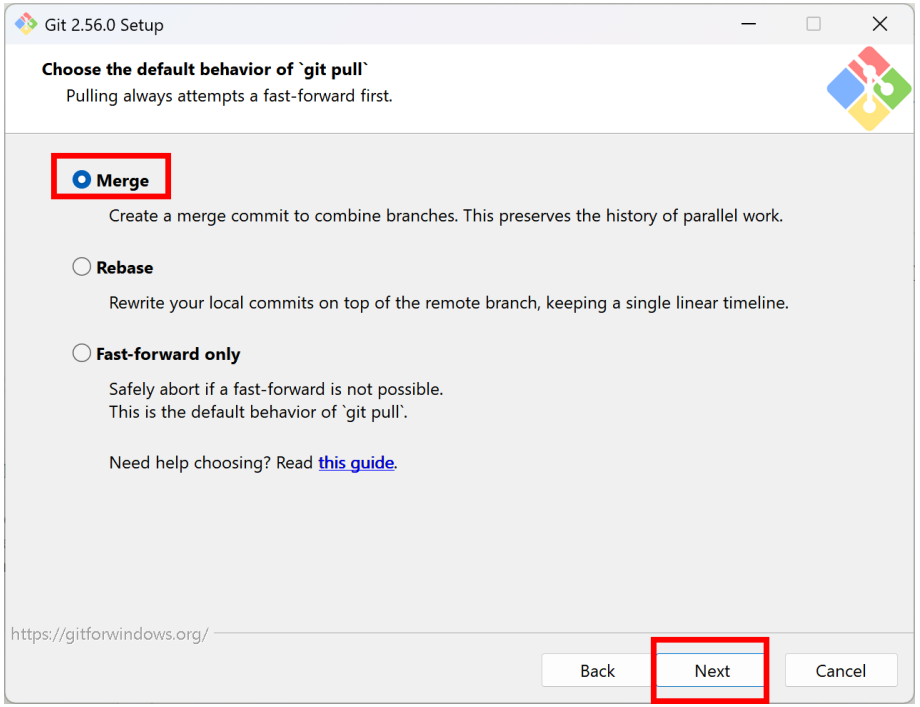

Git 2.56.0 Setup
Choosing the SSH executable
Which Secure Shell client program would you like Git to use?
Use bundled OpenSSH
This uses ssh.exe that comes with Git.
Use (Tortoise)Plink
To use PuTTY, specify the path to an existing copy of (Tortoise)Plink.exe:
...
Set ssh.variant for Tortoise Plink
Use external OpenSSH
This uses an external ssh.exe. Git will not install its own OpenSSH (and related) binaries but use them as found on the PATH.
https://gitforwindows.org/
Back
Next
Cancel


Git 2.56.0 Setup
Choosing HTTPS transport backend
Which SSL/TLS library would you like Git to use for HTTPS connections?
Use the OpenSSL library
Server certificates will be validated using the ca-bundle.crt file.
Use the native Windows Secure Channel library
Server certificates will be validated using Windows Certificate Stores.
This option also allows you to use your company's internal Root CA certificates
distributed e.g. via Active Directory Domain Services.
https://gitforwindows.org/
Back
Next
Cancel

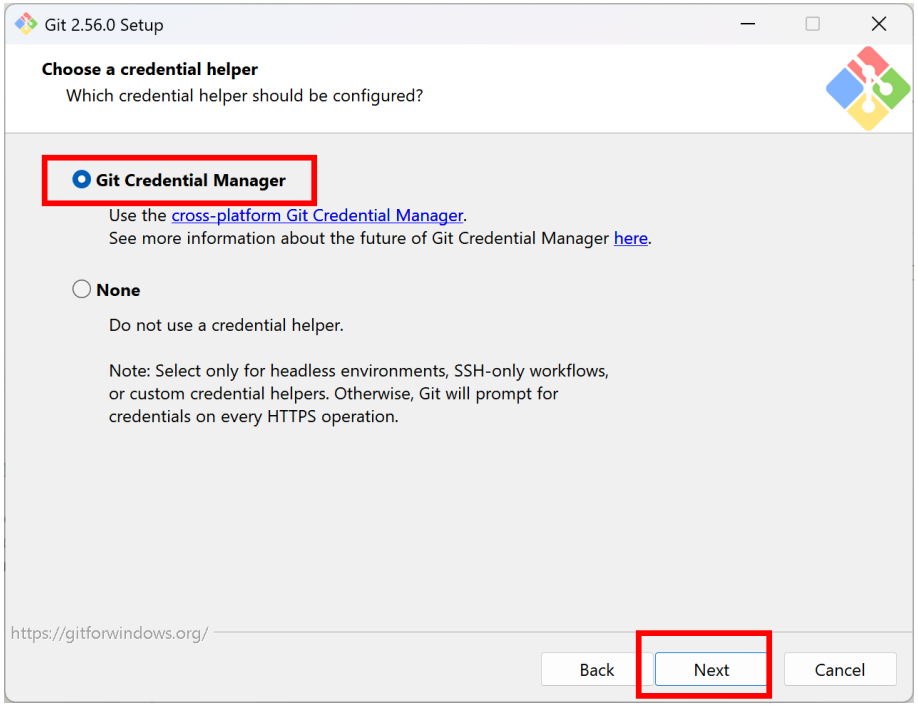

Git 2.56.0 Setup
Configuring the line ending conversions
How should Git treat line endings in text files?
Checkout Windows-style, commit Unix-style line endings
Git will convert LF to CRLF when checking out text files. When committing
text files, CRLF will be converted to LF. For cross-platform projects,
this is the recommended setting on Windows ("core.autocrlf" is set to "true").
Checkout as-is, commit Unix-style line endings
Git will not perform any conversion when checking out text files. When
committing text files, CRLF will be converted to LF. For cross-platform projects,
this is the recommended setting on Unix ("core.autocrlf" is set to "input").
Checkout as-is, commit as-is
Git will not perform any conversions when checking out or committing
text files. Choosing this option is not recommended for cross-platform
projects ("core.autocrlf" is set to "false").
https://gitforwindows.org/
Back
Next
Cancel

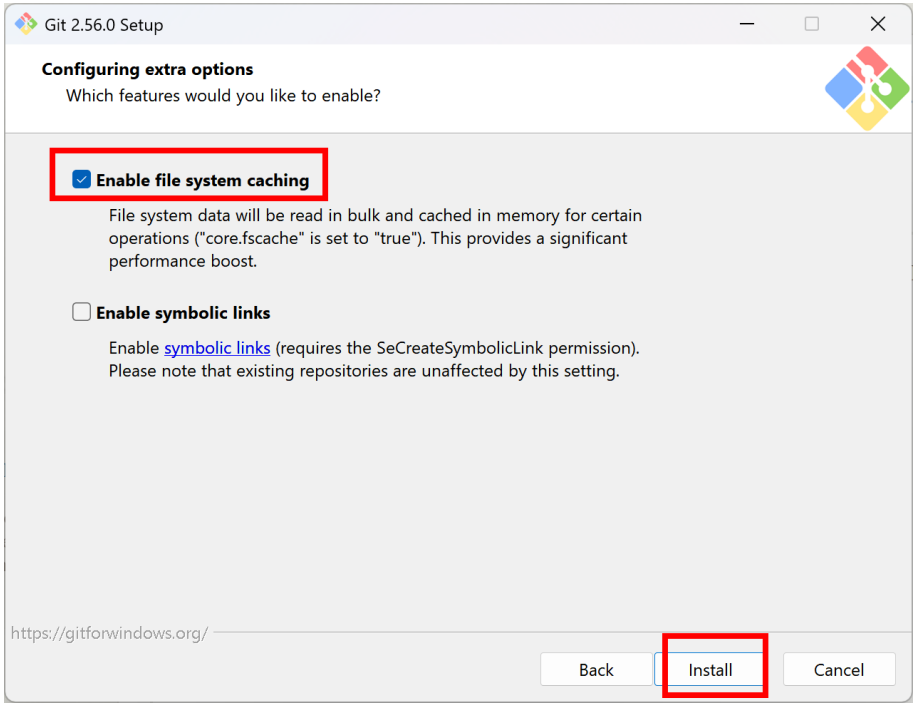
2. Git 설치


Git 2.56.0 Setup
Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?
Use MinTTY (the default terminal of MSYS2)
Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
Use Windows' default console window
Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.
https://gitforwindows.org/
Back
Next
Cancel

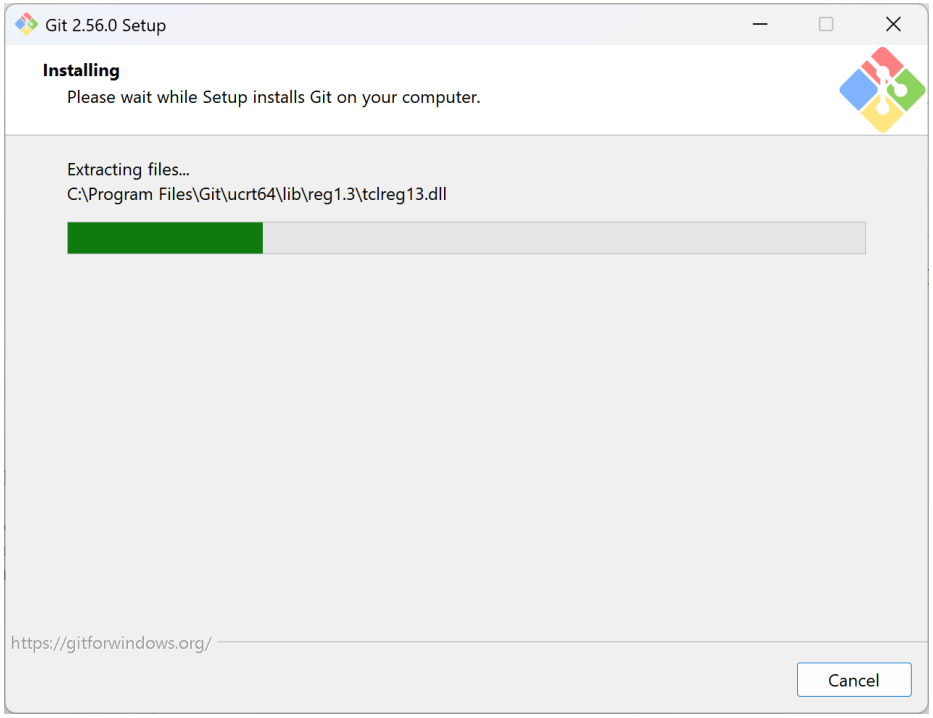

Git 2.56.0 Setup
Choose the default behavior of `git pull`
Pulling always attempts a fast-forward first.
Merge
Create a merge commit to combine branches. This preserves the history of parallel work.
Rebase
Rewrite your local commits on top of the remote branch, keeping a single linear timeline.
Fast-forward only
Safely abort if a fast-forward is not possible.
This is the default behavior of `git pull`.
Need help choosing? Read this guide.
https://gitforwindows.org/
Back
Next
Cancel

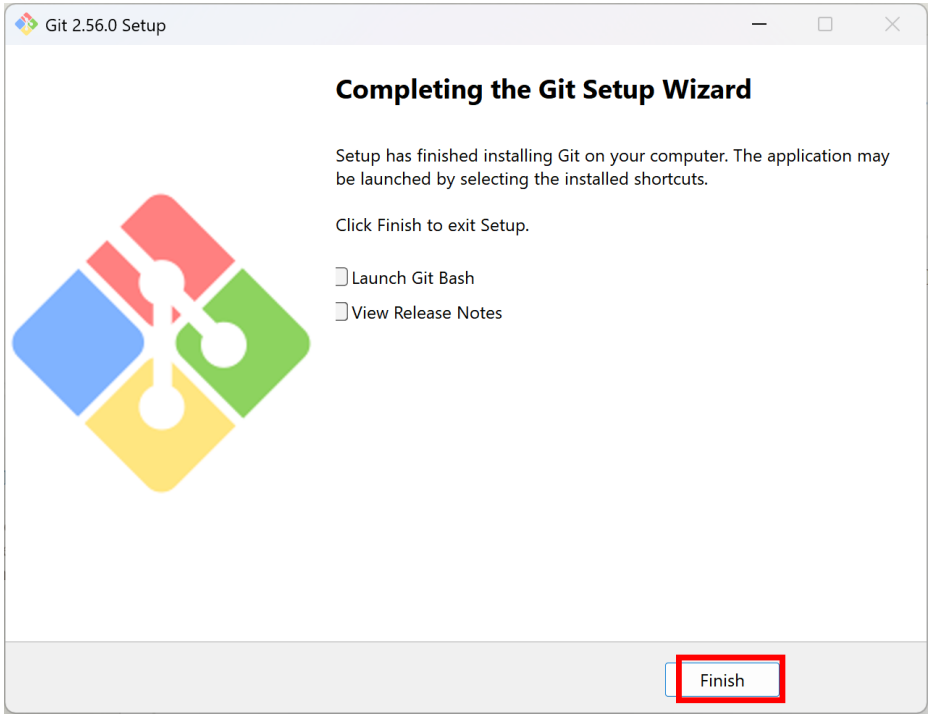
2. Git 설치


Git 2.56.0 Setup
Choose a credential helper
Which credential helper should be configured?
Git Credential Manager
Use the cross-platform Git Credential Manager.
See more information about the future of Git Credential Manager here.
None
Do not use a credential helper.
Note: Select only for headless environments, SSH-only workflows,
or custom credential helpers. Otherwise, Git will prompt for
credentials on every HTTPS operation.
https://gitforwindows.org/
Back
Next
Cancel


Git 2.56.0 Setup
Configuring extra options
Which features would you like to enable?
Enable file system caching
File system data will be read in bulk and cached in memory for certain
operations ("core.fscache" is set to "true"). This provides a significant
performance boost.
Enable symbolic links
Enable symbolic links (requires the SeCreateSymbolicLink permission).
Please note that existing repositories are unaffected by this setting.
https://gitforwindows.org/
Back
Install
Cancel

2. Git 설치


Git 2.56.0 Setup
Installing
Please wait while Setup installs Git on your computer.
Extracting files...
C:\Program Files\Git\ucrt64\lib\reg1.3\tclreg13.dll
https://gitforwindows.org/
Cancel


Git 2.56.0 Setup
Completing the Git Setup Wizard
Setup has finished installing Git on your computer. The application may be launched by selecting the installed shortcuts.
Click Finish to exit Setup.
Launch Git Bash
View Release Notes
Finish

2. Git 설치

```
C:\Users\User>git --version
git version 2.56.0.windows.1

C:\Users\User>
```

기본 editor를 VSC로 변경하기

1) window에서 path 경로 추가

PATH=%PATH%;C:₩Users₩계정명₩AppData₩Local₩Programs₩Microsoft VS Code₩bin;

2) git config --global core.editor "code --wait"

https://git-scm.com/book/en/v2/Appendix-C%3A-Git-Commands-Setup-and-Config 가서 참조.

3. 사용자 정보 등록 및 삭제

터미널에서 git의 버전을 저장할 때 사용될 사용자 정보(이름,이메일)를 설정한다.

https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup



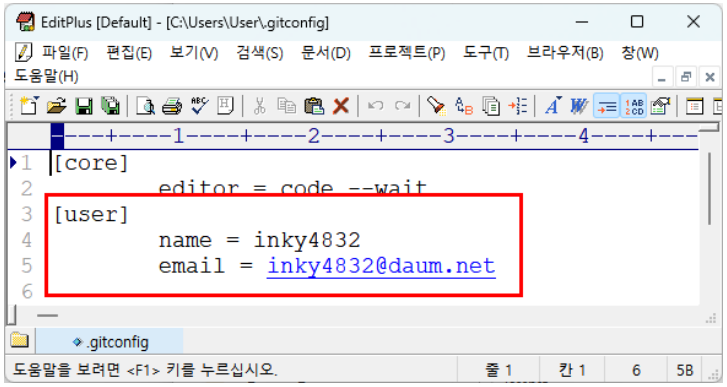
사용자 정보 설정

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config --global user.name "inky4832"

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config --global user.email "inky4832@daum.net"
```

C:₩Users₩계정명₩.gitconfig 파일이 생성되고 여기에 전역 사용자 정보가 저장된다.

이메일 정보는 github 계정에 로그인할 때 사용하는 이메일을 지정한다.



3. 사용자 정보 등록 및 삭제

전역 사용자 정보 확인

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config user.name
inky4832

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config user.email
inky4832@daum.net
```

전역 사용자 정보 삭제

```
inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config --global --unset user.name

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config user.name

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config --global --unset user.email

inky4@DESKTOP-KERHMJL MINGW64 ~
$ git config user.email

inky4@DESKTOP-KERHMJL MINGW64 ~
$
```

로컬 저장소 생성 및 관리

1. Git 명령어 reference

https://git-scm.com/docs

Reference

Quick reference guides: **GitHub Cheat Sheet** | **Visual Git Cheat Sheet**

Complete list of all commands

Setup and Config
- git
- config
- help
- bugreport
- Credential helpers

Getting and Creating Projects
- init
- clone

Basic Snapshotting
- add
- status
- diff
- commit
- notes
- restore
- reset
- rm
- mv

Guides
- gitattributes
- Command-line interface conventions
- Everyday Git
- Frequently Asked Questions (FAQ)
- Glossary
- Hooks
- gitignore
- gitmodules
- Revisions
- Submodules
- Tutorial
- Workflows
- All guides...

Email
- am
- apply
- format-patch
- send-email
- request-pull

Branching and Merging
- branch
- checkout
- switch
- merge
- mergetool
- log
- stash
- tag
- worktree

Sharing and Updating Projects
- fetch
- pull
- push
- remote
- submodule

Inspection and Comparison
- show
- log
- diff
- difftool
- range-diff
- shortlog
- describe

Patching
- apply
- cherry-pick
- diff
- rebase
- revert

Debugging
- bisect
- blame
- grep

External Systems
- svn
- fast-import

Administration
- clean
- gc
- fsck
- reflog
- filter-branch
- instaweb
- archive
- bundle

Server Admin
- daemon
- update-server-info

Plumbing Commands
- cat-file
- check-ignore
- checkout-index
- commit-tree
- count-objects
- diff-index
- for-each-ref
- hash-object
- ls-files
- ls-tree
- merge-base
- read-tree
- rev-list
- rev-parse
- show-ref
- symbolic-ref
- update-index
- update-ref
- verify-pack
- write-tree

2. 기본 명령어

git 기본 명령어

저장소 관리에 필요한 git 기본 명령어

명령어	기능	설명
git init	저장소 생성	실행한 위치를 git 저장소(로컬)로 지정.
git add 파일명	저장소에 파일추가	해당파일을 git이 추적할 수 있도록 저장소에 추가.
git commit	저장소에 수정내용 제출	변경된 파일을 저장소에 제출하면 새로운 버전이 생성.
git status	저장소 상태 확인	현재 저장소의 상태 출력.
git log	커밋 정보 확인	git이 관리하고 있는 commit된 정보 출력.

2. 로컬 저장소 생성

명령어: git init

https://git-scm.com/docs/git-init

NAME

git-init - Create an empty Git repository or reinitialize an existing one

SYNOPSIS

```
git init [-q | --quiet] [--bare] [--template=<template-directory>]
          [--separate-git-dir <git-dir>] [--object-format=<format>]
          [--ref-format=<format>]
          [-b <branch-name> | --initial-branch=<branch-name>]
          [--shared[=<permissions>]] [<directory>]
```

DESCRIPTION

This command creates an empty Git repository - basically a `.git` directory with subdirectories for `objects`, `refs/heads`, `refs/tags`, and template files. An initial branch without any commits will be created (see the `--initial-branch` option below for its name).

If the `$GIT_DIR` environment variable is set then it specifies a path to use instead of `./.git` for the base of the repository.

If the object storage directory is specified via the `$GIT_OBJECT_DIRECTORY` environment variable then the sha1 directories are created underneath; otherwise, the default `$GIT_DIR/objects` directory is used.

Running `git init` in an existing repository is safe. It will not overwrite things that are already there. The primary reason for rerunning `git init` is to pick up newly added templates (or to move the repository to another place if `--separate-git-dir` is given).

2. 로컬 저장소 생성

명령어: git init

저장소를 만들고 싶은 디렉토리로 이동해서 git을 초기화하면 그때부터 해당 디렉터리에 있는 파일들을 버전관리 할 수 있다.

현재 디렉토리 위치 확인

```
MINGW64:/c/Users/inky4

inky4@DESKTOP-410SFPS MINGW64 ~
$ pwd
/c/Users/inky4
```

디렉터리 생성 및 이동

git 저장소를 만들 디렉터리를 생성하고 해당 디렉터리로 이동

```
inky4@DESKTOP-410SFPS MINGW64 ~
$ mkdir git_tutorial

inky4@DESKTOP-410SFPS MINGW64 ~
$ cd git_tutorial/
```

2. 로컬 저장소 생성

git 저장소 생성

git init 명령어로 로컬 저장소를 생성하면 .git 숨김 폴더 및 main 브랜치가 생성된다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial
$ git init                                          .git 숨김폴더 생성
Initialized empty Git repository in C:/Users/inky4/git_tutorial/.git/

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)   main 브랜치
$
```

디렉터리 안의 내용을 확인하면 소스코드 버전이 저장될 저장소인 .git 디렉터리가 생성되어 있다. 이곳에 모든 버전 정보(이력)들이 저장되며 하위 디렉터리까지 포함해서 추적 관리하게 된다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ ls -al
total 16
drwxr-xr-x 1 inky4 197609 0 Jun  8 13:46 ./
drwxr-xr-x 1 inky4 197609 0 Jun  8 13:44 ../
drwxr-xr-x 1 inky4 197609 0 Jun  8 13:46 .git/
```

3. 로컬 저장소 상태 확인

명령어: git status

https://git-scm.com/docs/git-status

§ **NAME**

git-status - Show the working tree status

SYNOPSIS

```
git status [<options>] [--] [<pathspec>…]
```

DESCRIPTION

Displays paths that have differences between the index file and the current HEAD commit, paths that have differences between the working tree and the index file, and paths in the working tree that are not tracked by Git (and are not ignored by gitignore[5]). The first are what you `would` commit by running `git commit`; the second and third are what you `could` commit by running `git add` before running `git commit`.

3. 로컬 저장소 상태 확인

명령어: git status

저장소의 상태를 확인하기 위한 방법이다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git status
On branch main

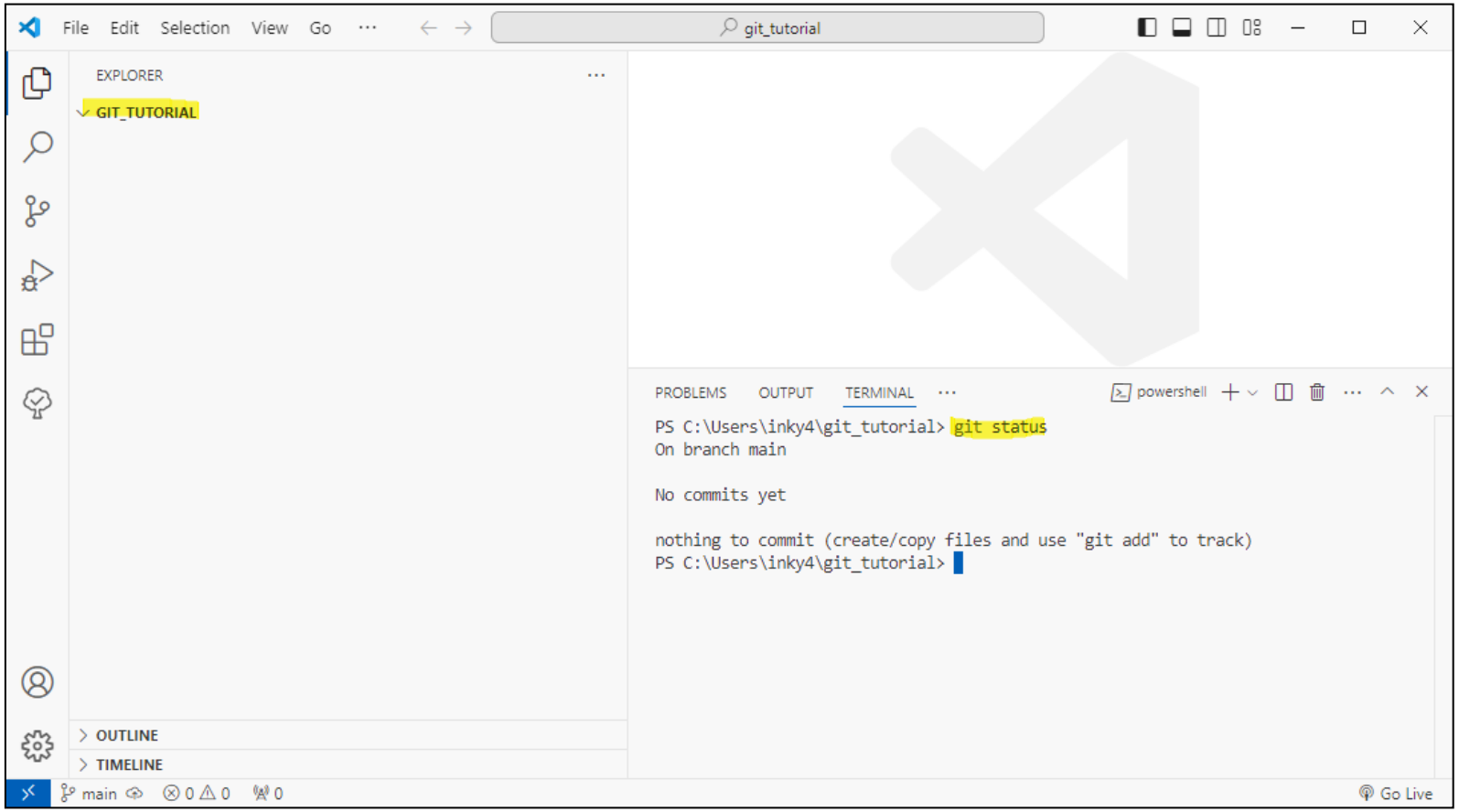
No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

위 상태 메시지의 의미는 다음과 같다.

1) 현재 main 브랜치에 있다.
2) 아직 커밋한 파일이 없다.
3) 현재 커밋할 파일이 없다.

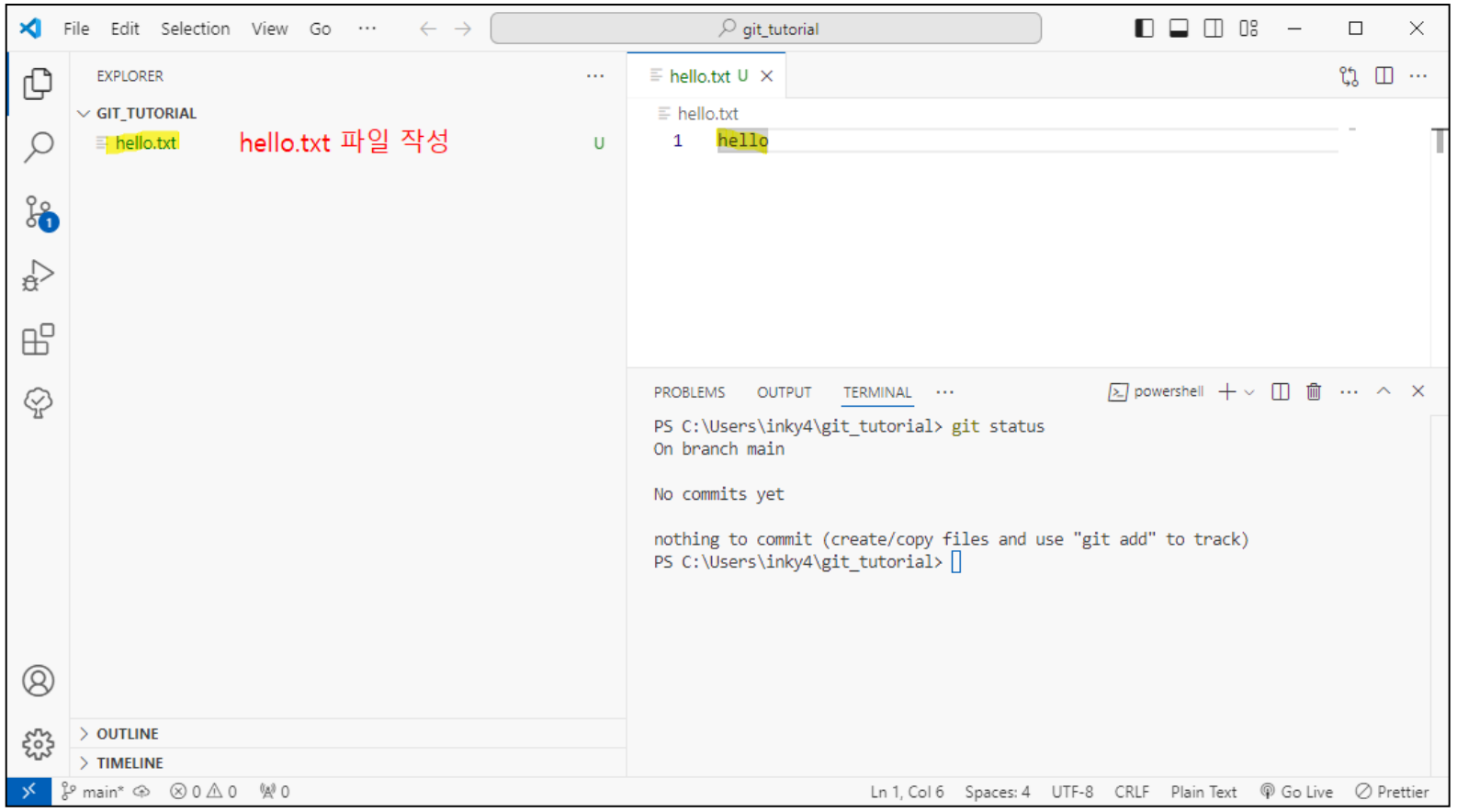
4. VSC에서 로컬 저장소 열기



5. 버전으로 관리할 문서 작성

문서 작성

VSC, 메모장 등 편집기를 통해서 hello.txt 파일을 생성하고 파일내부에 hello 문자열을 입력한 후 저장한다.



4. 버전으로 관리할 문서 작성

문서 작성 후 저장소 상태 확인

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git status
On branch main

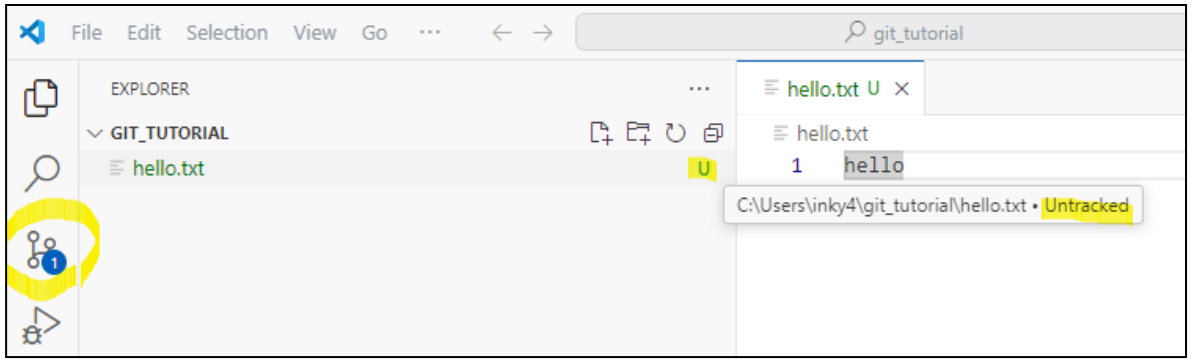
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Git 에서는 아직 한번도 버전관리 되지 않은 파일을 untracked files라고 한다.
즉 저장소에 git 이 아직 추적하지 않는 파일이 있음을 알려주는 화면이다.

다음은 VSC 에서 확인한 화면으로서 U 는 Untracked file 를 의미한다.



5. 추적 시킬 파일 등록

명령어: git add

https://git-scm.com/docs/git-add

§ NAME

git-add - Add file contents to the index

SYNOPSIS

```
git add [--verbose | -v] [--dry-run | -n] [--force | -f] [--interactive |
-i] [--patch | -p]
          [--edit | -e] [--[no-]all | -A | --[no-]ignore-removal | [--
update | -u]] [--sparse]
          [--intent-to-add | -N] [--refresh] [--ignore-errors] [--ignore-
missing] [--renormalize]
          [--chmod=(+|-)x] [--pathspec-from-file=<file> [--pathspec-file-
nul]]
          [--] [<pathspec>…]
```

DESCRIPTION

This command updates the index using the current content found in the working tree, to prepare the content staged for the next commit. It typically adds the current content of existing paths as a whole, but with some options it can also be used to add content with only part of the changes made to the working tree files applied, or remove paths that do not exist in the working tree anymore.

The "index" holds a snapshot of the content of the working tree, and it is this snapshot that is taken as the contents of the next commit. Thus after making any changes to the working tree, and before running the commit command, you must use the `add` command to add any new or modified files to the index.

This command can be performed multiple times before a commit. It only adds the content of the specified file(s) at the time the add command is run; if you want subsequent changes included in the next commit, then you must run `git add` again to add the new content to the index.

The `git status` command can be used to obtain a summary of which files have changes that are staged for the next commit.

The `git add` command will not add ignored files by default. If any ignored files were explicitly specified on the command line, `git add` will fail with a list of ignored files. Ignored files reached by directory recursion or filename globbing performed by Git (quote your globs before the shell) will be silently ignored. The `git add` command can be used to add ignored files with the `-f` (force) option.

Please see git-commit[1] for alternative ways to add content to a commit.

5. 추적 시킬 파일 등록

명령어: git add 파일명 [파일명2 …]
git add sub/파일명
git add .

git 에게 버전 만들 준비를 하라고 알려주는 작업으로서 스테이징(staging) 또는 '스테이지에 올린다' 또는 인덱스(index)에 등록한다 ' 라고 부른다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git add hello.txt

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$
```

```
$ git status
On branch master

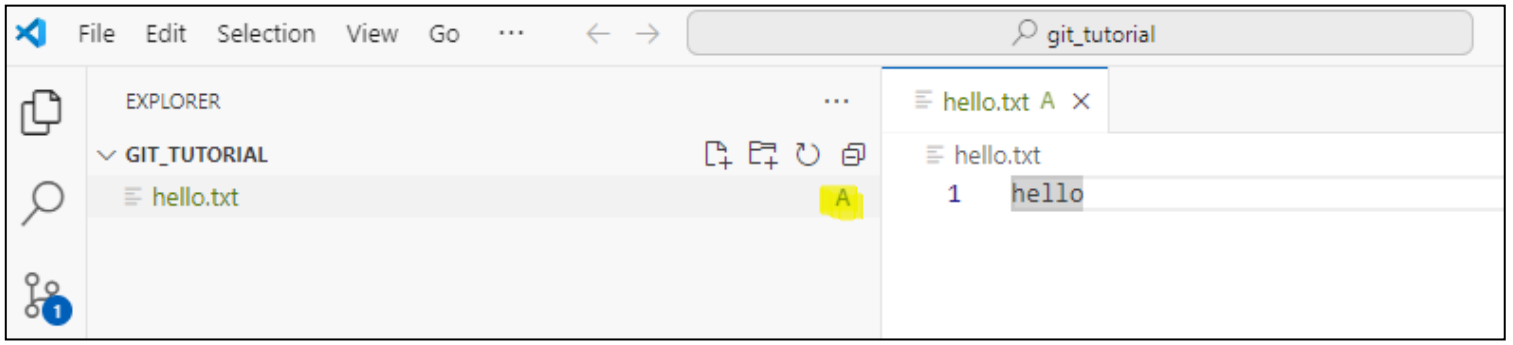
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.txt
```

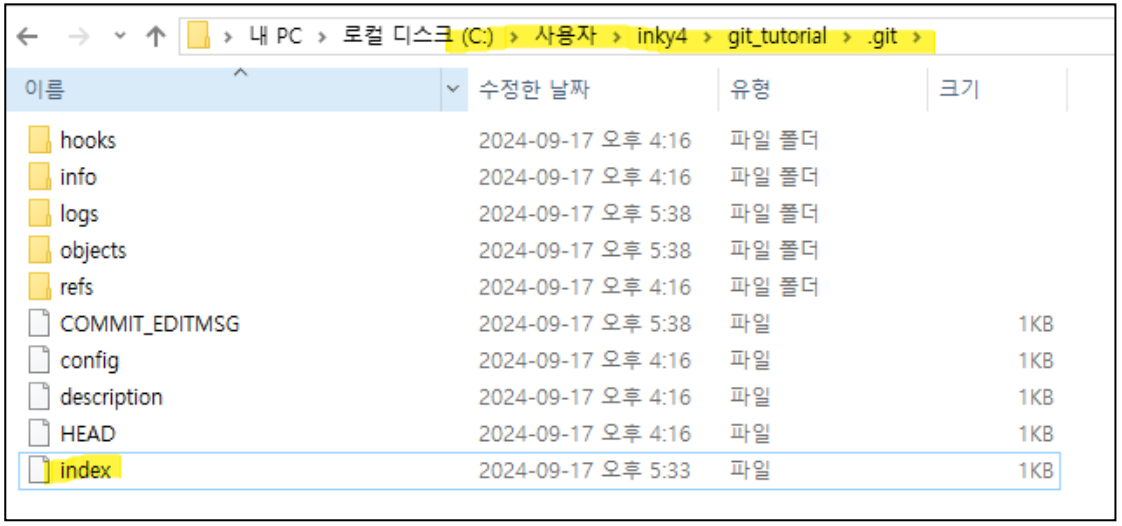
untracked files 문장이 changes to be committed로 바뀌었는데, 이것은 새로운 파일 hello.txt를 앞으로 커밋할 것이라는 의미이다.

5. 추적 시킬 파일 등록

다음은 VSC 에서 확인한 화면으로서 A는 Index Added 된 file 를 의미한다.



add 된 정보는 .git/index 파일에 저장된다. 바이너리 파일이기 때문에 확인은 불가.



명령어: git commit

https://git-scm.com/docs/git-commit

NAME

git-commit - Record changes to the repository

SYNOPSIS

```
git commit [-a | --interactive | --patch] [-s] [-v] [-u<mode>] [--amend]
           [--dry-run] [(-c | -C | --squash) <commit> | --fixup
[(amend|reword):]<commit>]
           [-F <file> | -m <msg>] [--reset-author] [--allow-empty]
           [--allow-empty-message] [--no-verify] [-e] [--author=<author>]
           [--date=<date>] [--cleanup=<mode>] [--[no-]status]
           [-i | -o] [--pathspec-from-file=<file> [--pathspec-file-nul]]
           [(--trailer <token>[(=|:)<value>])…] [-S[<keyid>]]
           [--] [<pathspec>…]
```

DESCRIPTION

Create a new commit containing the current contents of the index and the given log message describing the changes. The new commit is a direct child of HEAD, usually the tip of the current branch, and the branch is updated to point to it (unless no branch is associated with the working tree, in which case HEAD is "detached" as described in git-checkout[1]).

6. 버전 만들기

명령어: git commit –m “커밋 메시지 "

파일이 stage에 있으면 새로운 버전을 만들 수 있다.
git에서는 버전을 만드는 작업을 커밋(commit) 한다고 말한다.
커밋을 할 때는 그 버전에 어떤 변경사항이 있었는지 메시지를 함께 저장한다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git commit -m "first commit"
[main (root-commit) 601d1ca] first commit
 1 file changed, 1 insertion(+)
 create mode 100644 hello.txt
```

커밋 후의 결과 메시지를 보면 파일 1개가 변경되었고 파일 1개가 추가되었다.
즉 stage에 있던 hello.txt 파일이 저장소에 추가된 것이다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git status
On branch main
nothing to commit, working tree clean

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$
```

현재 git상태를 보면 버전으로 만들 파일이 없고 working tree(작업 디렉토리)도 수정사항 없이 깨끗하다는 메시지가 출력된다.

6. 버전 만들기

다음과 같이 hello2 를 추가하여 hello.txt 파일을 수정한다.
VSC 에서는 다음과 같이 hello.txt 파일 옆에 M 문자가 보이게 된다. (Modified 의미)



hello.txt 파일은 더 이상 untracked 파일이 아니기 때문에 다음과 같이 modified 가 출력된다.

```
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

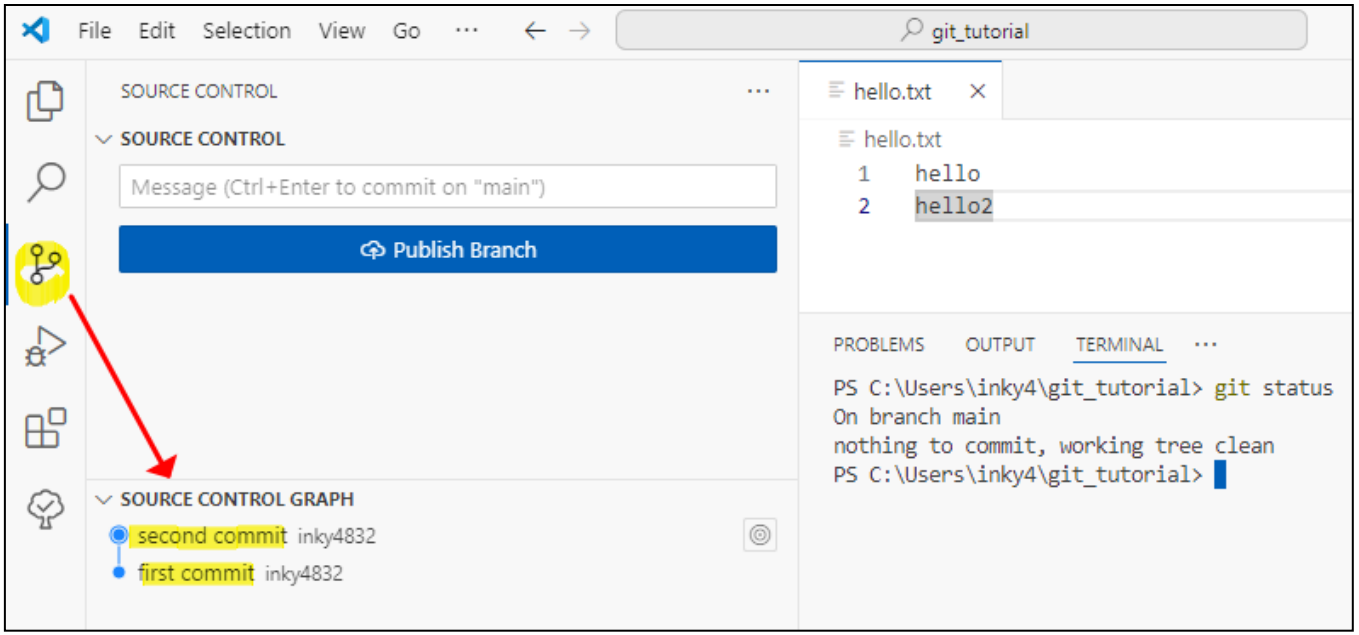
6. 버전 만들기

hello.txt 파일을 스테이징하기 위해서 git add 하고 커밋까지 진행한다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git add hello.txt
warning: in the working copy of 'hello.txt', LF will be r
ouches it

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git commit -m "second commit"
[main d32fee1] second commit
 1 file changed, 1 insertion(+)
```

다음은 VSC 에서 확인한 화면으로서 2 개의 버전이 관리되고 있음을 알 수 있다.



7. 버전 확인

명령어: git log

https://git-scm.com/docs/git-log

NAME

git-log - Show commit logs

SYNOPSIS

```
git log [<options>] [<revision-range>] [[--] <path>…]
```

DESCRIPTION

Shows the commit logs.

List commits that are reachable by following the `parent` links from the given commit(s), but exclude commits that are reachable from the one(s) given with a ^ in front of them. The output is given in reverse chronological order by default.

You can think of this as a set operation. Commits reachable from any of the commits given on the command line form a set, and then commits reachable from any of the ones given with ^ in front are subtracted from that set. The remaining commits are what comes out in the command's output. Various other options and paths parameters can be used to further limit the result.

7. 버전 확인

명령어 : git log 옵션

커밋(commit) 내역을 확인할 수 있는 명령어이다.

명령어	설명
git log	커밋한 버전에 대한 설명이 간략하게 출력
git log -p	각 커밋에 적용된 실제 변경 내용을 출력
git log --stat	각 커밋에서 수정된 파일의 통계정보 출력
git log --name-only	커밋 정보 중에서 수정된 파일의 목록만 출력
git log --relative-date	정확한 절대적인 시간이 아닌 상대적인 시간을 출력
git log --graph	브랜치 분기와 병합내역을 아스키 그래프로 출력
git log --oneline	한 줄에 한 커밋씩만 출력. 간략히 볼 때 사용

7. 버전 확인

git log 명령어

커밋한 버전에 대한 설명이 간략하게 출력된다.
커밋 해시값, 커밋을 만든 사람, 만든 시간, 커밋 메시지가 함께 출력된다.

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git log
commit 099b4626926fc83fc12f7898312f6923b96827bb (HEAD -> main)
Author: inky4832 <inky4832@daum.net>
Date:   Tue Sep 17 20:49:41 2024 +0900

    second commit

commit 17e3445f73944f1c61f1c7f3e5906c154a1a7cb8
Author: inky4832 <inky4832@daum.net>
Date:   Tue Sep 17 17:35:15 2024 +0900

    first commit

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
```

가장 최신 버전

커밋 해시값

7. 버전 확인

git log -p 명령어

각 커밋에서 적용된 실제 변경 내용을 출력

```
$ git log -p
commit 099b4626926fc83fc12f7898312f6923b96827bb (HEAD -> main)
Author: inky4832 <inky4832@daum.net>
Date:   Tue Sep 17 20:49:41 2024 +0900

    second commit

diff --git a/hello.txt b/hello.txt
index b6fc4c6..0acca82 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1 +1,2 @@
-hello
\ No newline at end of file
+hello
+hello2
\ No newline at end of file

commit 17e3445f73944f1c61f1c7f3e5906c154a1a7cb8
Author: inky4832 <inky4832@daum.net>
Date:   Tue Sep 17 17:35:15 2024 +0900

    first commit

diff --git a/hello.txt b/hello.txt
new file mode 100644
index 0000000..b6fc4c6
--- /dev/null
+++ b/hello.txt
@@ -0,0 +1 @@
+hello
```

7. 버전 확인

git log --relative-date 명령어

현재 시간 기준으로 commit 시간이 얼마나 경과되었는지를 출력.

```
$ git log --relative-date
commit 099b4626926fc83fc12f7898312f6923b96827bb (HEAD -> main)
Author: inky4832 <inky4832@daum.net>
Date:   2 hours ago

    second commit

commit 17e3445f73944f1c61f1c7f3e5906c154a1a7cb8
Author: inky4832 <inky4832@daum.net>
Date:   5 hours ago

    first commit
```

7. 버전 확인

git log --graph 명령어

브랜치 분기와 병합내역을 아스키 그래프로 출력.

```
$ git log --graph
* commit 099b4626926fc83fc12f7898312f6923b96827bb (HEAD -> main)
| Author: inky4832 <inky4832@daum.net>
| Date:   Tue Sep 17 20:49:41 2024 +0900
|
|     second commit
|
* commit 17e3445f73944f1c61f1c7f3e5906c154a1a7cb8
  Author: inky4832 <inky4832@daum.net>
  Date:   Tue Sep 17 17:35:15 2024 +0900

      first commit
```

git log --oneline 명령어

커밋 정보를 매우 간략하게 한 줄로 출력.

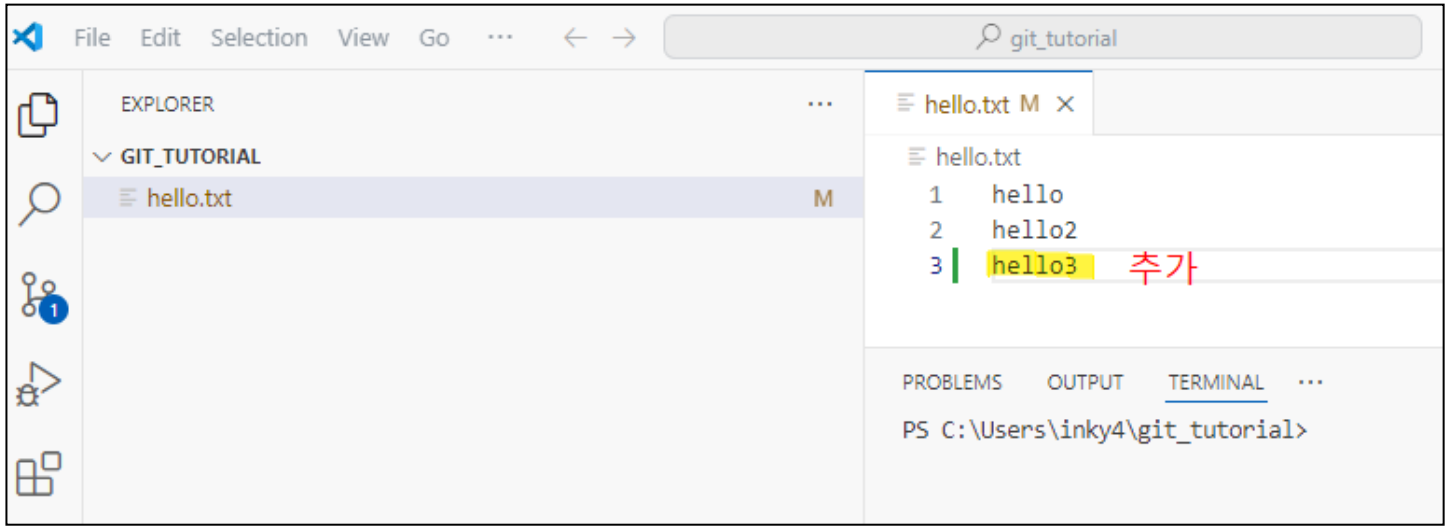
```
$ git log --oneline
099b462 (HEAD -> main) second commit
17e3445 first commit
```

8. 스테이징과 commit 한꺼번에

명령어 : git commit -am " 메시지"

commit 명령어에 -am 옵션을 사용하면 파일을 스테이지에 올리고 커밋하는 과정을 한 번에 처리할 수 있다.

단 반드시 한번이라도 커밋한 적이 있어야 된다.




```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git commit -am "third commit"
[main 4e10a59] third commit
 1 file changed, 2 insertions(+), 1 deletion(-)

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$
```

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git log --oneline
4e10a59 (HEAD -> main) third commit
099b462 second commit
17e3445 first commit
```

Git 저장소 구조
및
버전 단계마다의 파일 상태

1. 저장소 구조

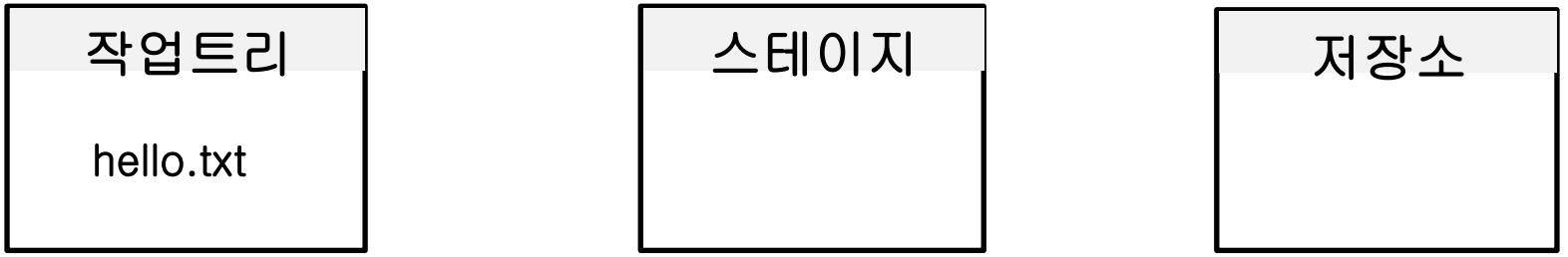
git은 관리할 파일들의 이름을 그대로 유지하면서 수정 내역을 기록한다.
이를 위해서 다음과 같은 구조를 사용한다.



1. 저장소 구조

스테이지와 저장소는 눈에 보이지 않는다.
.git 디렉터리안에 숨은 파일 형태로 존재하게 된다.

순서1) git init 으로 초기화하고 hello.txt 문서 작성한 후



git init

```
$ git status
On branch master

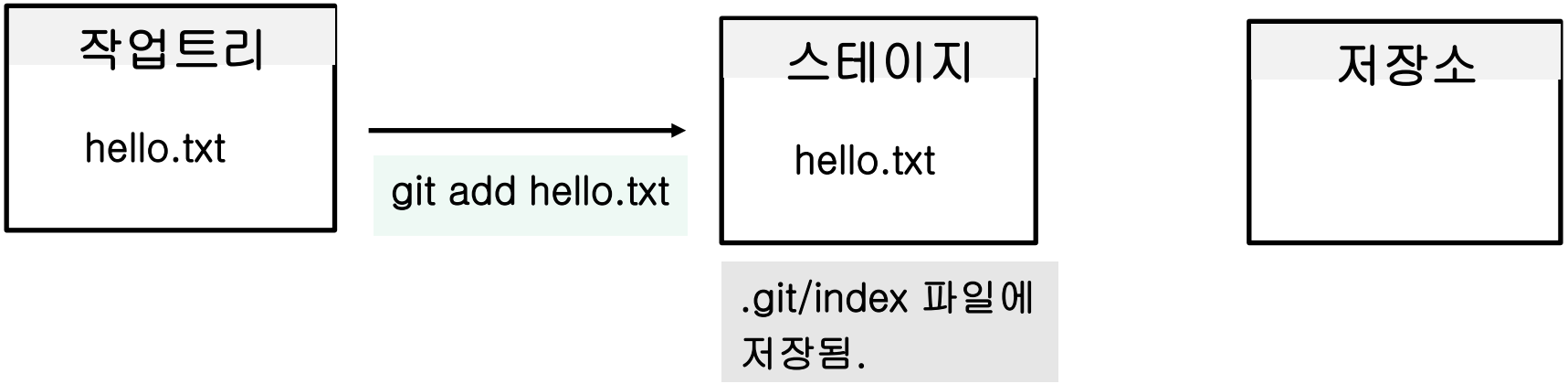
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.txt

nothing added to commit but untracked files present (use "git add" to track)

inky4@DESKTOP-410SFPS MINGW64 ~/git_tutorial (master)
```

순서 2) git add hello.txt 명령어로 추적파일로 지정한 후



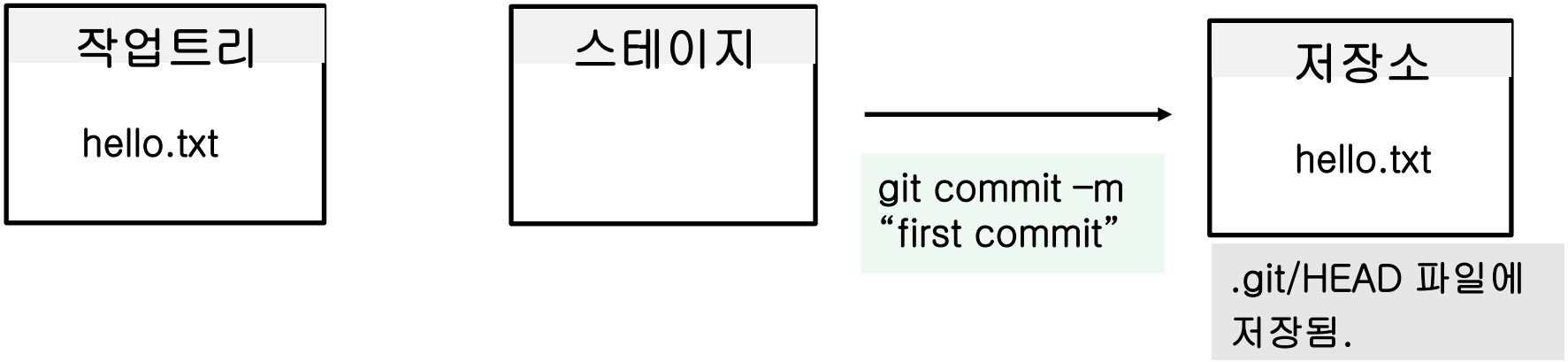
.git/index 파일에 저장됨.

```
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   hello.txt
```

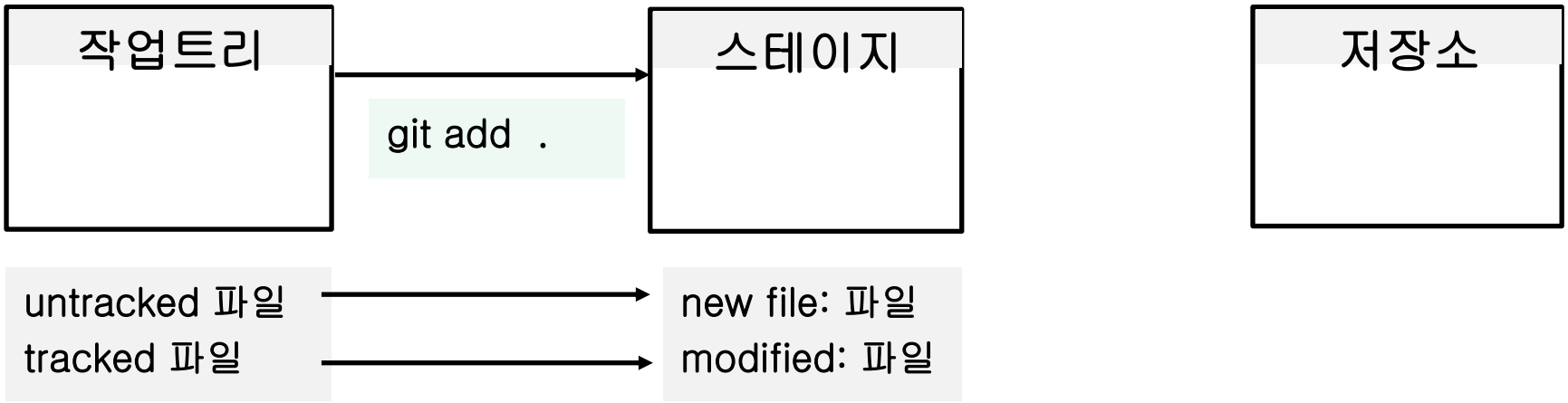
순서 3) git commit –m “first commit” 명령어로 저장소에 제출한 후




```
inky4@DESKTOP-410SFPS MINGW64 ~/git_tutorial (master)
$ git status
On branch master
nothing to commit, working tree clean
```

2. 버전 단계마다의 파일 상태

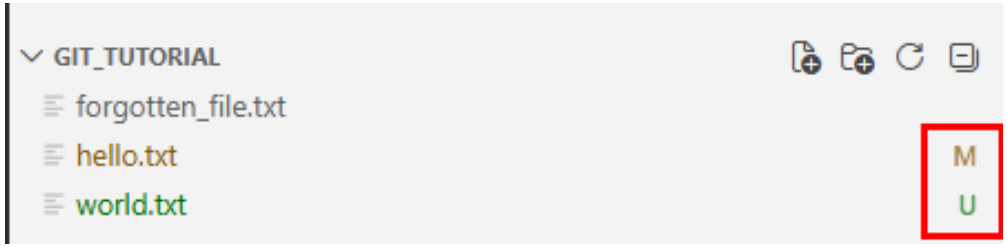
git 에서는 버전을 만드는 각 단계마다 파일 상태를 다르게 표시한다.
따라서 파일의 상태를 이해하면 이 파일이 버전 관리의 여러 단계 중에서 어디에 있는지 알 수 있다.



3. tracked vs untracked 파일

untracked 파일은 한번도 git에서 버전관리를 하지 않은 파일을 의미한다.
tracked 파일은 git이 한번이라도 커밋을 한 파일의 수정여부를 계속 추적하는 파일을 의미한다.

1) 기존 hello.txt 파일 수정하고 새로운 world.txt 파일 추가



작업트리
hello.txt world.txt

3. Tracked vs untracked 파일

2) 파일 상태 확인

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        world.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

hello.txt 파일은 changes not staged for commit 로서 변경된 파일이 스테이지에 올라가지 않았음을 알 수 있고 modified: 라고 표시되어 있어서 파일이 수정되었음을 알 수 있다. 이렇게 git은 한번이라도 커밋한 파일의 수정여부를 계속 추적하며 git이 추적하고 있다는 뜻에서 tracked 파일이라고 부른다.

반면에 world.txt 파일은 한번도 git이 버전관리를 하지 않았고 수정내역을 추적하지 않았기 때문에 untracked files 라고 출력된다.

3. Tracked vs untracked 파일

3) new file 및 modified 파일

```
$ git add .
warning: in the working copy of 'world.txt', LF will be re
t time Git touches it

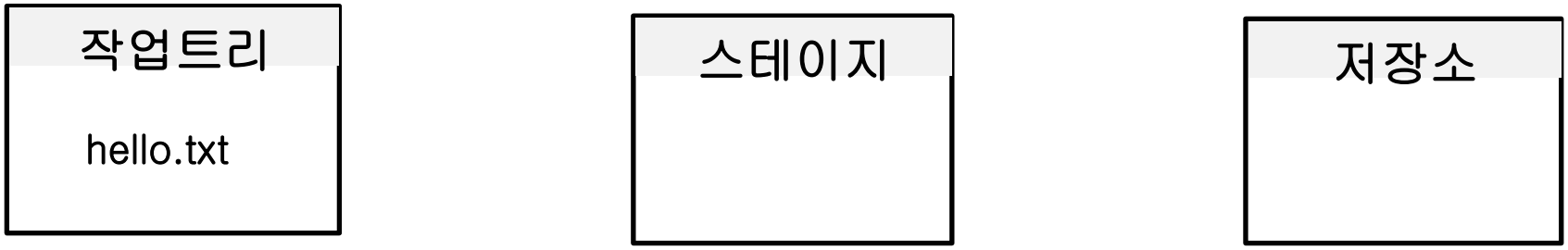
inky4@DESKTOP-410SFPS MINGW64 ~/git_tutorial (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   hello.txt
        new file:   world.txt
```

```
$ git commit -m 'new file world.txt'
[master 49d0628] new file world.txt
 2 files changed, 2 insertions(+)
 create mode 100644 world.txt
```

마지막 커밋 이후 수정된 hello.txt 파일(tracked)은 modified: 로 표시되고
버전 관리가 처음인 world.txt 파일(untracked)은 new file:로 표시된다.

4. Tracked 파일의 상태 (unmodified, modified, staged)

tracked 파일은 작업 트리에 있는지 스테이지에 있는지 등 더 구체적인 상태를 알 수 있다.



1) 현재 수정사항이 없는 경우

```
$ git status
On branch master
nothing to commit, working tree clean
```

unmodified 상태

4. Tracked 파일의 상태 (unmodified, modified, staged)

2) 현재 수정사항이 있는 경우 (world.txt 내용 수정후)

```
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   world.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

modified 상태

3) 스테이징 (git add 후)

```
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   world.txt
```

staged 상태

변경사항 비교

명령어: git diff

https://git-scm.com/docs/git-diff

NAME

git-diff - Show changes between commits, commit and working tree, etc

SYNOPSIS

```
git diff [<options>] [<commit>] [--] [<path>…]
git diff [<options>] --cached [--merge-base] [<commit>] [--] [<path>…]
git diff [<options>] [--merge-base] <commit> [<commit>…] <commit> [--]
[<path>…]
git diff [<options>] <commit>…<commit> [--] [<path>…]
git diff [<options>] <blob> <blob>
git diff [<options>] --no-index [--] <path> <path>
```

DESCRIPTION

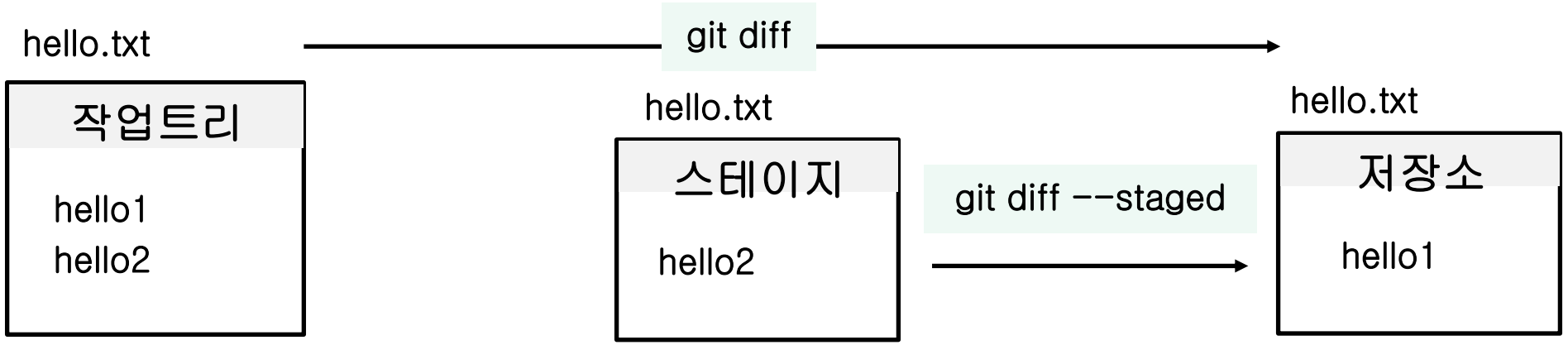
Show changes between the working tree and the index or a tree, changes between the index and a tree, changes between two trees, changes resulting from a merge, changes between two blob objects, or changes between two files on disk.

1. 변경사항 비교

명령어: git diff 옵션

git diff 명령어를 사용하면 파일의 어떤 내용이 변경되었는지 차이점을 비교 할 수 있다.

명령어	설명
git diff	commit 된 파일상태와 현재 수정중인 상태 비교
git diff --staged	commit 된 파일상태와 add된 파일 상태 비교
git diff HEAD	가장 최신 commit과 이후의 모든 변경사항 비교
git diff 해쉬값1 해쉬값2	commit간 상태 비교
git diff HEAD HEAD^	가장 최근의 commit과 그 바로 전(HEAD^)의 commit 비교
git diff branch1 branch2	branch 간의 상태 비교 예> git diff feature/main origin/main



2. git diff

git diff 명령어를 사용하면 수정내용이 포함된 작업 트리에 있는 파일과 최신 커밋 내용을 비교해서 수정파일을 커밋하기 전에 최종적으로 검토할 수 있다.

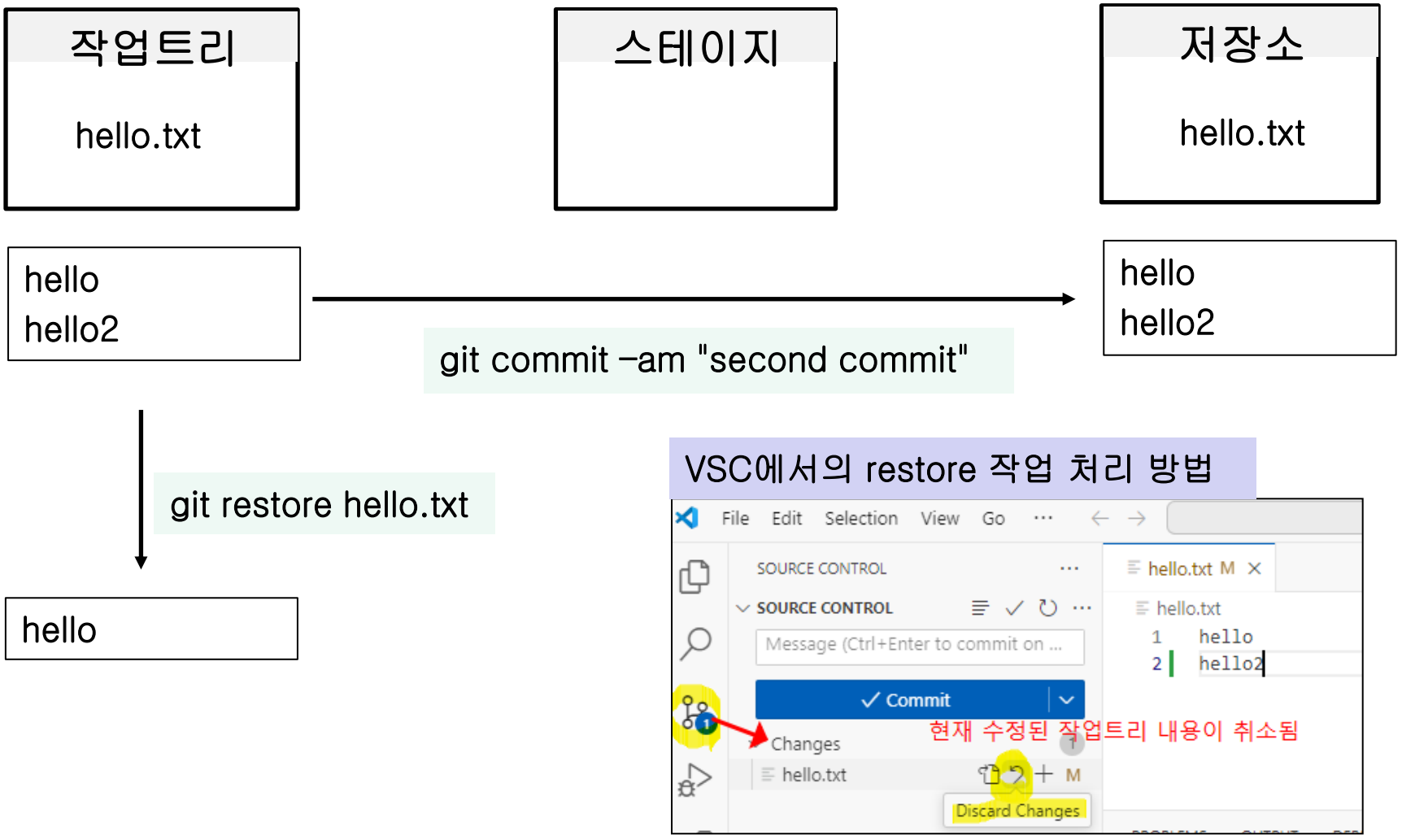



```
$ git diff
diff --git a/hello.txt b/hello.txt
index b6fc4c6..0acca82 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1 +1,2 @@
-hello
\ No newline at end of file
+hello
+hello2
\ No newline at end of file

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$
```

2. git diff

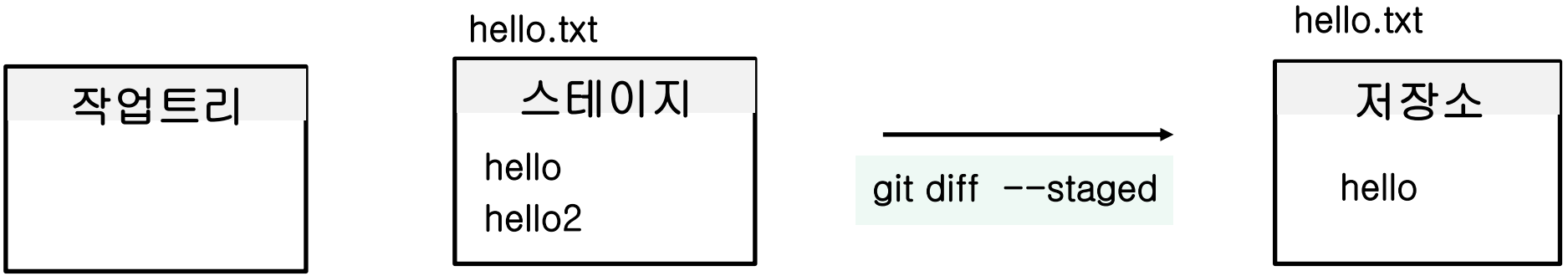
비교후 최종처리 작업 (커밋 또는 무시)



VSC에서의 restore 작업 처리 방법

3. git diff --staged

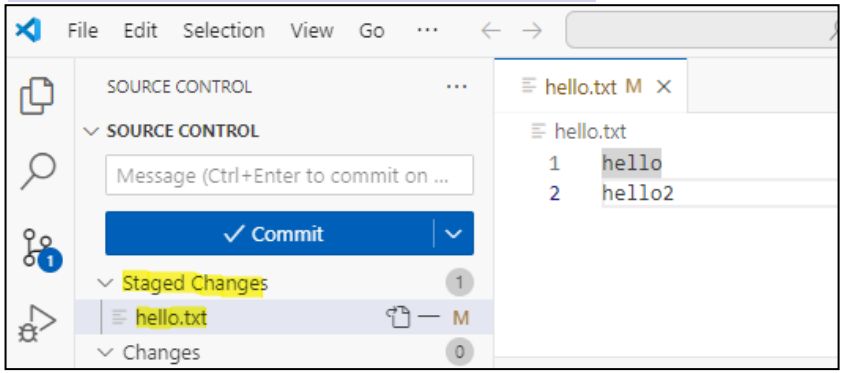
git diff --staged 명령어를 사용하면 stage에 있는 파일과 최신 커밋 내용을 비교해서 수정파일을 커밋하기 전에 최종적으로 검토할 수 있다.




```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git diff

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git diff --staged
diff --git a/hello.txt b/hello.txt
index b6fc4c6..0acca82 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1 +1,2 @@
-hello
\ No newline at end of file
+hello
+hello2
\ No newline at end of file
```

VSC에서의 현재 stage 상태



3. git diff --staged

비교 후 최종처리 작업 (커밋 또는 무시)

git commit -m "second commit"

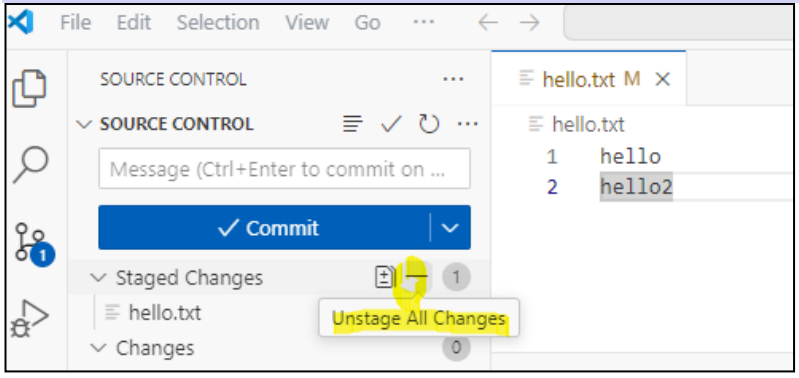



```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git restore --staged hello.txt

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git diff --staged

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
```

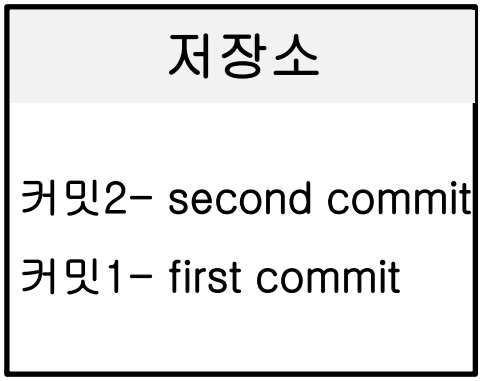
VSC에서의 restore --stagged 작업 처리 방법



4. git diff 해시값1 해시값2

git diff 해시값1 해시값2 이용하면 커밋간에 변경사항 목록을 확인할 수 있다.

hello.txt




```
$ git log --oneline
2f0140b (HEAD -> main) second commit
5b193a1 first commit

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git diff 5b193a1  2f0140b
diff --git a/hello.txt b/hello.txt
index b6fc4c6..0acca82 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1 +1,2 @@
-hello
\ No newline at end of file
+hello
+hello2
\ No newline at end of file

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git diff HEAD^  HEAD
diff --git a/hello.txt b/hello.txt
index b6fc4c6..0acca82 100644
--- a/hello.txt
+++ b/hello.txt
@@ -1 +1,2 @@
-hello
\ No newline at end of file
+hello
+hello2
\ No newline at end of file
```

불필요한 파일 및 폴더 무시

1. .gitignore 파일

https://git-scm.com/docs/gitignore

NAME

gitignore - Specifies intentionally untracked files to ignore

SYNOPSIS

$XDG_CONFIG_HOME/git/ignore, $GIT_DIR/info/exclude, .gitignore

DESCRIPTION

A `gitignore` file specifies intentionally untracked files that Git should ignore. Files already tracked by Git are not affected; see the NOTES below for details.

Each line in a `gitignore` file specifies a pattern. When deciding whether to ignore a path, Git normally checks `gitignore` patterns from multiple sources, with the following order of precedence, from highest to lowest (within one level of precedence, the last matching pattern decides the outcome):

- Patterns read from the command line for those commands that support them.
- Patterns read from a `.gitignore` file in the same directory as the path, or in any parent directory (up to the top-level of the working tree), with patterns in the higher level files being overridden by those in lower level files down to the directory containing the file. These patterns match relative to the location of the `.gitignore` file. A project normally includes such `.gitignore` files in its repository, containing patterns for files generated as part of the project build.
- Patterns read from `$GIT_DIR/info/exclude`.
- Patterns read from the file specified by the configuration variable `core.excludesFile`.

1. .gitignore 파일

개요

버전 관리중인 디렉터리안에 버전 관리를 하지 않을 특정 파일이나 폴더가 있다면 .gitignore 파일을 만들고 그 안에 파일 또는 폴더명을 지정하면 된다.

샘플

```
HELP.md
target/
!.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/

### STS ###
.apt_generated
.classpath
.factorypath
.project
.settings
.springBeans
.sts4-cache

### IntelliJ IDEA ###
.idea
*.iws
*.iml
*.ipr
```

```
### NetBeans ###
/nbproject/private/
/nbbuild/
/dist/
/nbdist/
/.nb-gradle/
build/
!**/src/main/**/build/
!**/src/test/**/build/

### VS Code ###
.vscode/


.metadata/

### Environment variables ###
.env
```

2. .gitignore 파일 작성

1) 현재 디렉터리 목록보기

```
$ ls -al
total 21
drwxr-xr-x 1 inky4 197609  0 Jun  8 23:46 ./
drwxr-xr-x 1 inky4 197609  0 Jun  8 23:54 ../
drwxr-xr-x 1 inky4 197609  0 Jun  8 23:52 .git/
-rw-r--r-- 1 inky4 197609 16 Jun  8 23:46 hello.txt
```

2) tmp 폴더와 Test.java 파일 생성 (tmp 폴더에 반드시 exam.txt 파일 추가)

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ mkdir tmp

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ touch Test.java
```

2. .gitignore 파일 작성

3) 디렉터리 목록보기

```
$ ls -al
total 21
drwxr-xr-x 1 inky4 197609  0 Jun  9 00:02 ./
drwxr-xr-x 1 inky4 197609  0 Jun  8 23:54 ../
drwxr-xr-x 1 inky4 197609  0 Jun  8 23:52 .git/
-rw-r--r-- 1 inky4 197609  0 Jun  9 00:02 Test.java
-rw-r--r-- 1 inky4 197609 16 Jun  8 23:46 hello.txt
drwxr-xr-x 1 inky4 197609  0 Jun  9 00:02 tmp/
```

4) 상태정보 확인

```
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Test.java
        tmp/

nothing added to commit but untracked files present (use "git add" to track)
```

2. .gitignore 파일 작성

5) .gitignore 파일 작성



5) 상태정보 확인

```
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

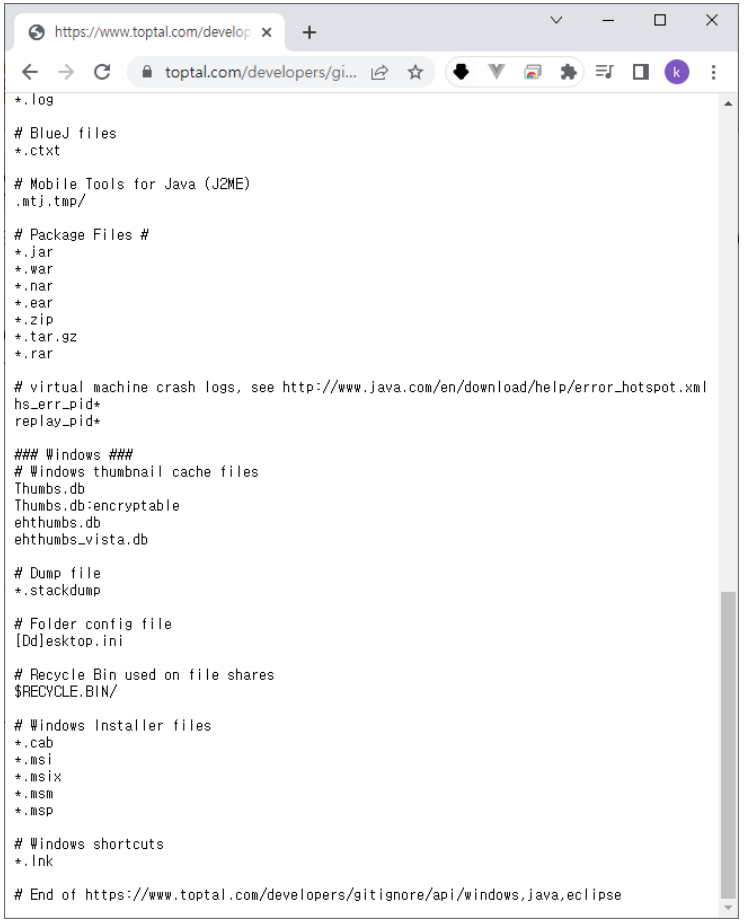
nothing added to commit but untracked files present (use "git add" to track)
```

.gitignore 파일안에 명시된 Test.java 와 tmp 폴더가 버전 관리 목록에서 제외된 것을 확인할 수 있다.

3. gitignore.io 웹 사이트 이용

https://www.toptal.com/developers/gitignore



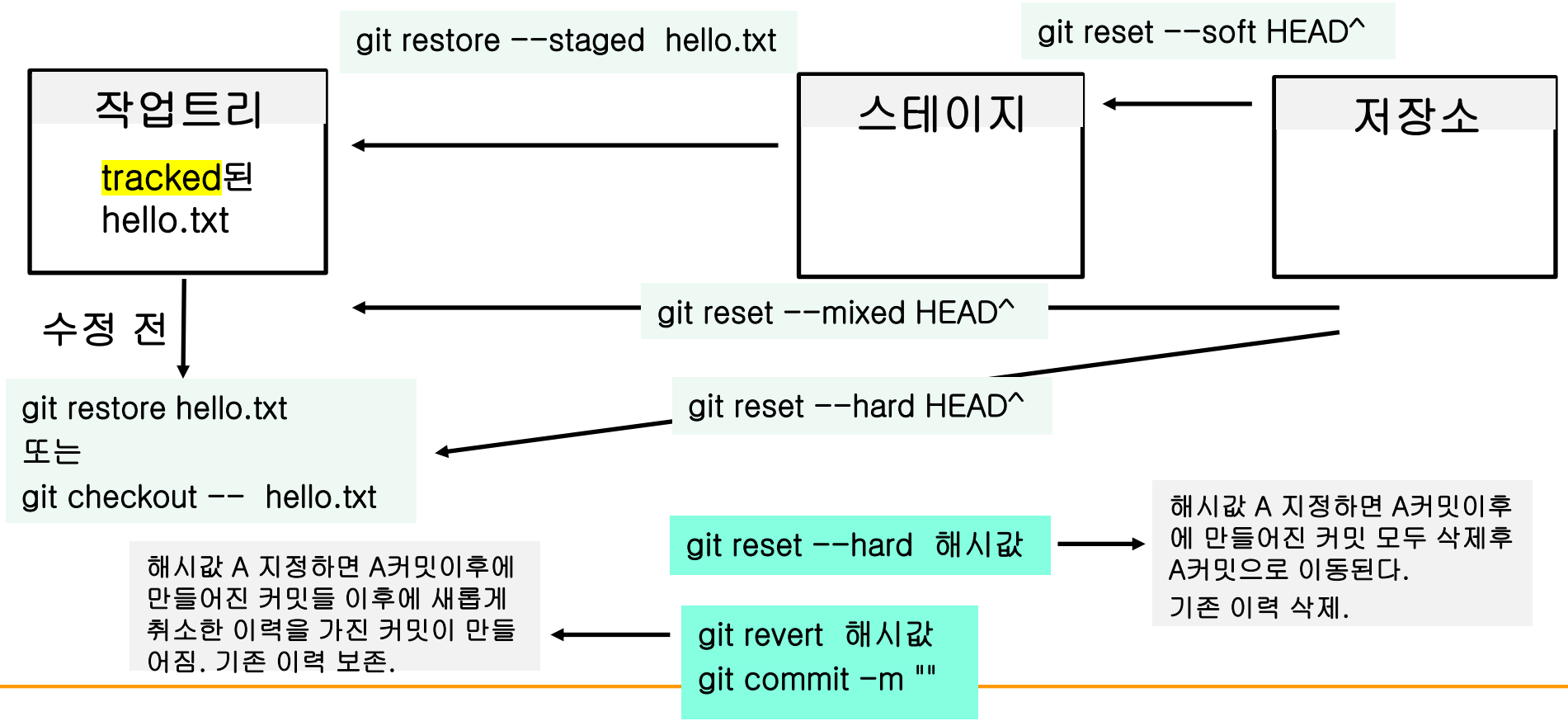


작업 되돌리기

1. 작업 되돌리기

작업 트리에서 수정했던 변경사항을 취소하거나 스테이지에 올렸던 파일을 내리거나 커밋을 취소하는 등 각 단계에서 수행했던 작업을 되돌리는 방법이다.

주의할 점은 반드시 tracked 된 파일이어야 된다.



작업 되돌리기 1

(git restore)

2. 작업 트리 또는 스테이지에서 수정한 파일 되돌리기

명령어: git restore 파일명

https://git-scm.com/docs/git-restore

§ NAME

git-restore - Restore working tree files

SYNOPSIS

```
git restore [<options>] [--source=<tree>] [--staged] [--worktree] [--]
<pathspec>…
git restore [<options>] [--source=<tree>] [--staged] [--worktree] --
pathspec-from-file=<file> [--pathspec-file-nul]
git restore (-p|--patch) [<options>] [--source=<tree>] [--staged] [--
worktree] [--] [<pathspec>…]
```

DESCRIPTION

Restore specified paths in the working tree with some contents from a restore source. If a path is tracked but does not exist in the restore source, it will be removed to match the source.

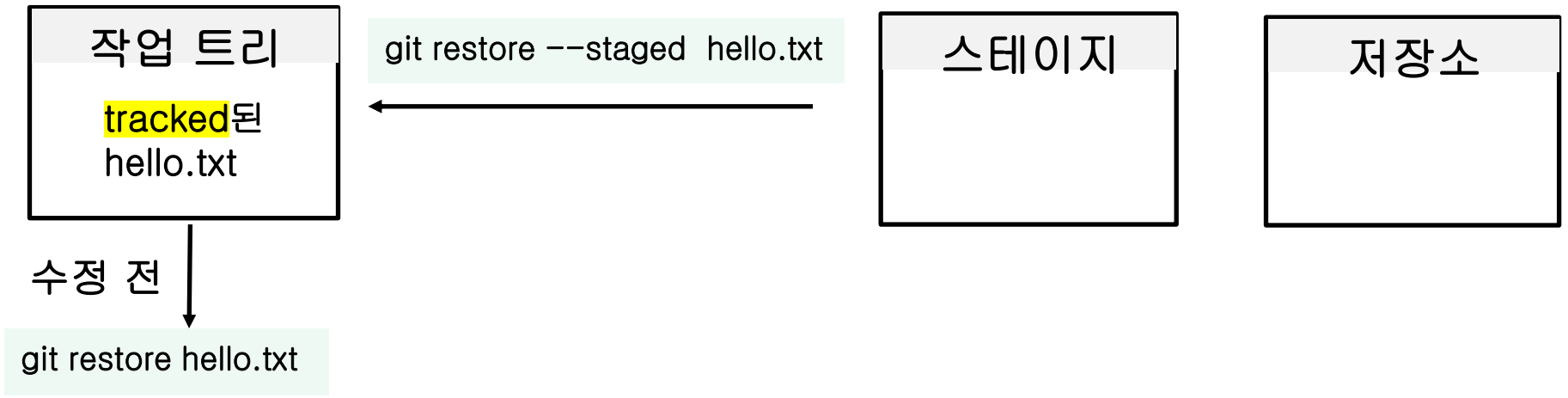
The command can also be used to restore the content in the index with `--staged`, or restore both the working tree and the index with `--staged --worktree`.

By default, if `--staged` is given, the contents are restored from `HEAD`, otherwise from the index. Use `--source` to restore from a different commit.

See "Reset, restore and revert" in git[1] for the differences between the three commands.

THIS COMMAND IS EXPERIMENTAL. THE BEHAVIOR MAY CHANGE.

2. 작업 트리 또는 스테이지에서 수정한 파일 되돌리기



git restore hello.txt 실습

```
$ mkdir git_restore
$ cd git_restore
$ git init
$ touch hello.txt  ( 반드시 tracked 파일이어야  revert가 가능 )
$ 파일 수정
$ git add hello.txt
$ git commit -m 'first commit'
$ git status

$ 파일 수정만 ( add 안함 )
$ git restore hello.txt  (작업트리 수정사항 되돌리기 )
$ git status
```

git restore --staged hello.txt 실습

```
$ touch a.txt b.txt  c.txt
$ 파일 수정
$ git add .      ( commit  안함 )
$ git status
# c.txt 는 스테이지에서 제외 시킴
$ git restore --staged c.txt
$ git status     (c.txt 파일은 더 이상 추적되지 않음)
```

작업 되돌리기 2

(git reset)

3. 특정 커밋으로 재설정

명령어: git reset 해시값

https://git-scm.com/docs/git-reset

NAME

git-reset - Reset current HEAD to the specified state

SYNOPSIS

```
git reset [-q] [<tree-ish>] [--] <pathspec>…
git reset [-q] [--pathspec-from-file=<file> [--pathspec-file-nul]] [<tree-ish>]
git reset (--patch | -p) [<tree-ish>] [--] [<pathspec>…]
git reset [--soft | --mixed [-N] | --hard | --merge | --keep] [-q] [<commit>]
```

DESCRIPTION

In the first three forms, copy entries from `<tree-ish>` to the index. In the last form, set the current branch head (`HEAD`) to `<commit>` , optionally modifying index and working tree to match. The `<tree-ish>` / `<commit>` defaults to `HEAD` in all forms.

git reset [-q] [<tree-ish>] [--] <pathspec>…
git reset [-q] [--pathspec-from-file=<file> [--pathspec-file-nul]] [<tree-ish>]

These forms reset the index entries for all paths that match the `<pathspec>` to their state at `<tree-ish>` . (It does not affect the working tree or the current branch.)

This means that `git reset <pathspec>` is the opposite of `git add <pathspec>` . This command is equivalent to `git restore [--source=<tree-ish>] --staged <pathspec>...` .

저장소를 특정 커밋으로 재설정한다.

이때 이전 커밋을 완전히 제거하고 브랜치 포인터를 뒤로 이동시킨다.

즉 이전 커밋이 처음부터 없었던 것처럼 동작된다.

3. 특정 커밋으로 재설정

```
git reset HEAD^
git reset HEAD~3
git reset –soft    HEAD^|해시값
git reset –mixed HEAD^|해시값      (기본)
git reset –hard    HEAD^|해시값
```

명령어	설명
--soft HEAD^	최근 커밋을 하기 전 스테이지 상태로 되돌리기
--mixed HEAD^	최근 커밋과 스테이징 하기 전 상태로 되돌리기. 기본(default)
--hard HEAD^	최근 커밋,스테이징,파일수정 하기 전 상태로 되돌리기

§ `--soft`

Does not touch the index file or the working tree at all (but resets the head to `<commit>`, just like all modes do). This leaves all your changed files "Changes to be committed", as `git status` would put it.

`--mixed`

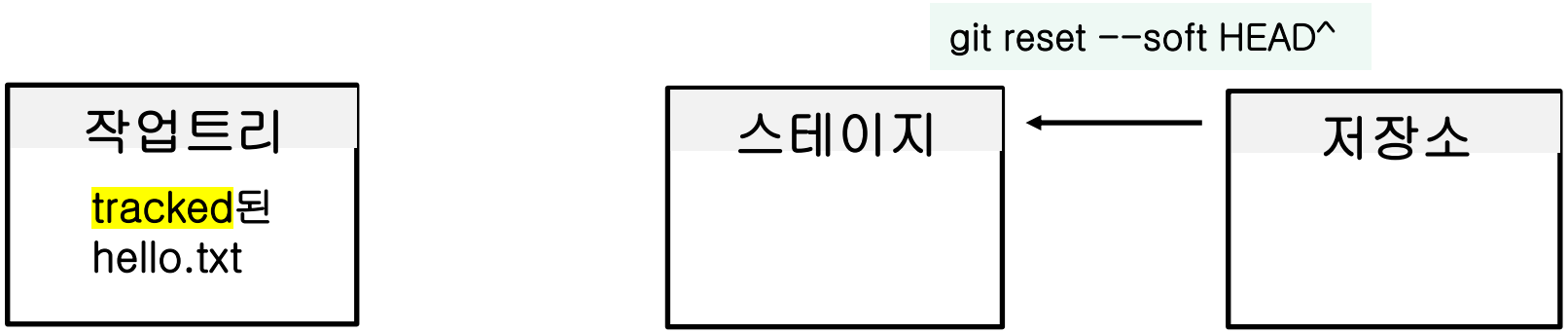
Resets the index but not the working tree (i.e., the changed files are preserved but not marked for commit) and reports what has not been updated. This is the default action.

If `-N` is specified, removed paths are marked as intent-to-add (see git-add[1]).

`--hard`

Resets the index and working tree. Any changes to tracked files in the working tree since `<commit>` are discarded. Any untracked files or directories in the way of writing any tracked files are simply deleted.

3. 특정 커밋으로 재설정



git reset --soft HEAD^|해시값

```
$ mkdir git_reset
$ cd git_reset
$ git init
$ touch hello.txt
# 파일수정
$ git add .
$ git commit -m "hello1 commit"
# 2번 정도 commit 작업 추가 ( "hello2 commit" , "hello3 commit" )
$ git log -oneline
$ git reset --soft HEAD^          (가장 최신 커밋 취소. 즉 hello3 commit 이 취소됨.)
$ git log -oneline
 $ git status
On branch main
Changes to be committed:
(use "git restore --staged <file>..." to unstage)
                modified:   hello.txt
```

3. 특정 커밋으로 재설정



git reset --mixed HEAD ^|해시값

```
$ mkdir git_reset2
$ cd git_reset2
$ git init
$ touch hello.txt
# 파일수정
$ git add .
$ git commit -m "hello1 commit"
# 2번 정도 commit 작업 추가 ( "hello2 commit" , "hello3 commit" )
$ git log -oneline
$ git status
$ git reset --mixed HEAD^          (가장 최신 커밋 취소. 즉 hello3 commit 이 취소됨.)
$ git log -oneline
$ git status
On branch main
Changes not staged for commit:
        (use "git add <file>..." to update what will be committed)
        (use "git restore <file>..." to discard changes in working directory)
                modified:   hello.txt
```

3. 특정 커밋으로 재설정



git reset --hard HEAD ^|해시값

```
$ mkdir git_reset3
$ cd git_reset3
$ git init
$ touch hello.txt
# 파일수정
$ git add .
$ git commit -m "hello1 commit"
# 2번 정도 commit 작업 추가 ( "hello2 commit" , "hello3 commit" )
$ git log -oneline
$ git status
$ git reset --hard HEAD^          (가장 최신 커밋 취소. 즉 hello3 commit 이 취소됨.)
$ git log -oneline
$ git status
On branch main
 On branch main
                  nothing to commit, working tree clean
```

작업 되돌리기 3

(git revert)

4. 기존 커밋 삭제 않고 특정 커밋으로 재설정

명령어: git revert 해시값

https://git-scm.com/docs/git-revert

NAME

git-revert - Revert some existing commits

SYNOPSIS

```
git revert [--[no-]edit] [-n] [-m <parent-number>] [-s] [-S[<keyid>]]
<commit>…
git revert (--continue | --skip | --abort | --quit)
```

DESCRIPTION

Given one or more existing commits, revert the changes that the related patches introduce, and record some new commits that record them. This requires your working tree to be clean (no modifications from the HEAD commit).

Note: `git revert` is used to record some new commits to reverse the effect of some earlier commits (often only a faulty one). If you want to throw away all uncommitted changes in your working directory, you should see git-reset[1], particularly the `--hard` option. If you want to extract specific files as they were in another commit, you should see git-restore[1], specifically the `--source` option. Take care with these alternatives as both will discard uncommitted changes in your working directory.

See "Reset, restore and revert" in git[1] for the differences between the three commands.

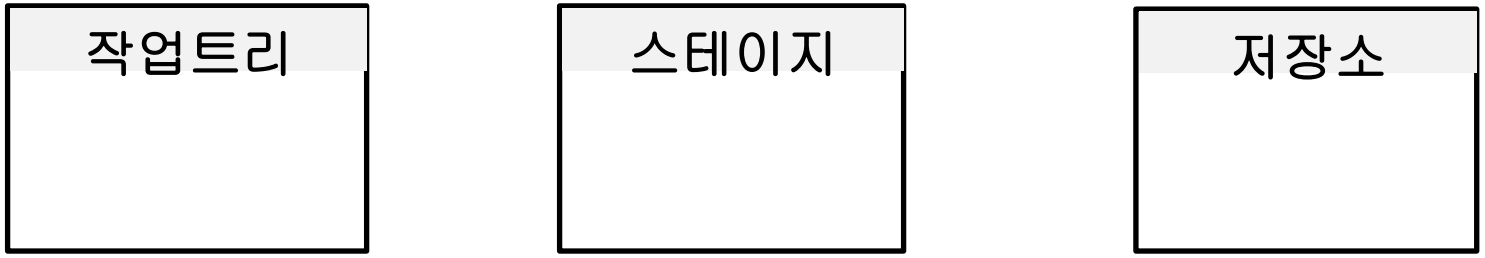
기존 commit 시점으로 가는 것이 아니고 기존 시점의 새로운 commit을 생성함.

따라서 기존 commit 이력은 유지됨. (권장)

새로운 commit 작성을 위한 commit 메시지 입력창이 보여지고 일반적으로 충돌이 발생된다.

이유는 저장소의 내용 및 취소 내용과 병합과정에서 충돌이 발생되기 때문이다.

4. 기존 커밋 삭제 않고 특정 커밋으로 재설정



해시값 A 지정하면 A 커밋이후에 만들어진 커밋들 이후에 새롭게 취소한 이력을 가진 커밋이 만들어짐. 기존 이력 보존.

git revert 해시값

git revert 해시값

```
$ mkdir git_revert
$ cd git_revert
$ git init
$ touch hello.txt
# 파일수정
$ git add .
$ git commit -m "hello1 commit"
# 2번정도 commit 작업
$ git log -oneline
7d16736 (HEAD -> main) hello3 commit
8a0c1f2 hello2 commit
7089331 hello1 commit
```

```
 $ git revert 8a0c1f2
CONFLICT (modify/delete): hello.txt deleted…
hint: After resolving the conflicts, mark them with
hint: "git add/rm <pathspec>", then run
hint: "git revert --continue".
// hint 참고해서 진행한다.
# 충돌된 경우 충돌된 hello.txt 문서 수정
$ git add .
$ git revert --continue    <== 커밋 메시지 지정 위한 새로운 창이 나옴.
                               커밋 메시지 작성하고 파일 저장.
$ git log --oneline
   c76aa68 (HEAD -> main) Revert "revert hello2 commit"
   7d16736 hello3 commit
   8a0c1f2 hello2 commit
   7089331 hello1 commit
```

브랜치 (branch)

1. 브랜치

개요

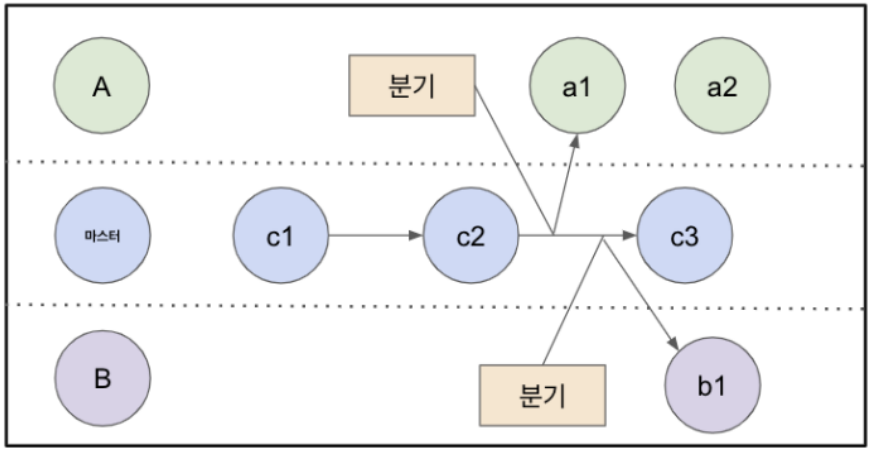
기존 파일에 새로운 기능을 추가하고자 하는 등, 제대로 동작하는 소스는 그대로 둔 채 새로운 소스를 추가한 버전을 따로 만들어 관리하는 경우에 사용된다.

기능

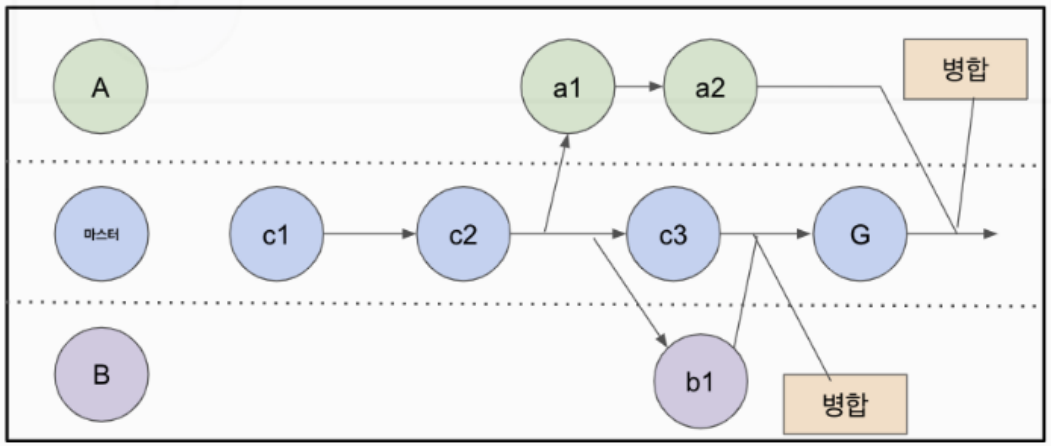
Git 으로 버전 관리를 시작하면 기본적으로 main브랜치가 만들어진다.

사용자가 커밋할 때마다 main브랜치는 최신 커밋을 가리킨다.

이때 새로운 브랜치를 만들면 기존에 저장한 파일을 main 브랜치에 그대로 유지하면서 기존 파일 내용을 수정하거나 새로운 기능을 구현할 파일을 만들 수 있다.



분기했던 새로운 브랜치에서 원하는 작업을 모두 끝낸 후 새로운 브랜치에 있던 파일을 원래 main 브랜치와 합칠수도 있다. (병합: merge)



2. 브랜치 기본 명령어

브랜치 관리에 필요한 git 기본 명령어

명령어	기능
git branch	브랜치 목록 보기
git branch 이름	지정된 이름의 새로운 브랜치 생성하기 -m 옵션 지정은 브랜치 이름 변경 작업
git branch -d 이름	병합후에 기존 브랜치 삭제하기 (-d 는 --delete --force의 축약어)
git switch 이름 git checkout 이름	브랜치 변경(이동)하기
git merge 이름	현재 작업중인 브랜치에서 지정된 이름의 브랜치를 가져와서 병합하기
git log --oneline --branches	모든 branch 커밋 이력을 한 줄로 출력하기
git log main..hotfix	main 브랜치 기준으로 hotfix 브랜치와의 차이점 출력

명령어: git branch

https://git-scm.com/docs/git-branch

§ NAME

git-branch - List, create, or delete branches

SYNOPSIS

```
git branch [--color[=<when>] | --no-color] [--show-current]
        [-v [--abbrev=<n> | --no-abbrev]]
        [--column[=<options>] | --no-column] [--sort=<key>]
        [--merged [<commit>]] [--no-merged [<commit>]]
        [--contains [<commit>]] [--no-contains [<commit>]]
        [--points-at <object>] [--format=<format>]
        [(-r | --remotes) | (-a | --all)]
        [--list] [<pattern>…]
git branch [--track[=(direct|inherit)] | --no-track] [-f]
        [--recurse-submodules] <branchname> [<start-point>]
git branch (--set-upstream-to=<upstream> | -u <upstream>) [<branchname>]
git branch --unset-upstream [<branchname>]
git branch (-m | -M) [<oldbranch>] <newbranch>
git branch (-c | -C) [<oldbranch>] <newbranch>
git branch (-d | -D) [-r] <branchname>…
git branch --edit-description [<branchname>]
```

1) git_tutorial 저장소 브랜치 목록 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch
* main
```

2) hotfix 브랜치 생성

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch hotfix

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch
  hotfix
* main
```

* 는 현재 사용중인 브랜치를 의미한다.

3) 브랜치 변경

main 브랜치에서 hotfix 브랜치로 변경한다. (git checkout hotfix 동일)

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git switch hotfix
Switched to branch 'hotfix'

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git branch
* hotfix
  main
```



지금부터 하는 모든 작업은 hotfix 브랜치만 영향을 미친다. (파일 수정, 삭제 등)
마음껏 작업하고 commit한 후에 main 브랜치로 switch하면, hotfix에서 했던 모든 작업을 해당 브랜치의 최종 commit 상태로 보존한 후에 main 브랜치의 최종 커밋 상태로 파일들이 변경된다.

4) hotfix 브랜치에서 파일 수정

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git branch
* hotfix
  main

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ cat hello.txt
hello
hello2
```

hello.txt 파일에 hello3 hotfix 문자열 입력하고 commit 하기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ cat hello.txt
hello
hello2
hello3 hotfix 추가하기
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git commit -am "hello3 hotfix commit"
[hotfix 8e09b2f] hello3 hotfix commit
 1 file changed, 2 insertions(+), 1 deletion(-)
```

hello3 hotfix 문자열 추가는 hotfix 브랜치에만 영향이 미치고 main 브랜치에는 영향이 없다.

5) hotfix 브랜치에서 커밋 이력

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git log --oneline
8e09b2f (HEAD -> hotfix) hello3 hotfix commit
7506f4d (main) gitignore 추가
2f0140b second commit
5b193a1 first commit
```

6) main 브랜치 기준으로 hotfix 브랜치와의 차이점 출력

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git log main..hotfix  main 브랜치가 기준
commit 8e09b2fda6ce5588ac4aa9bda64d9d1ffa7fcbd4 (HEAD -> hotfix)
Author: inky4832 <inky4832@daum.net>
Date:   Fri Sep 20 11:43:51 2024 +0900

    hello3 hotfix commit
```

main 브랜치에는 없고 hotfix 브랜치에만 있는 커밋 정보 출력.

명령어: git merge

https://git-scm.com/docs/git-merge

NAME

git-merge - Join two or more development histories together

SYNOPSIS

```
git merge [-n] [--stat] [--no-commit] [--squash] [--[no-]edit]
        [--no-verify] [-s <strategy>] [-X <strategy-option>] [-S[<keyid>]]
        [--[no-]allow-unrelated-histories]
        [--[no-]rerere-autoupdate] [-m <msg>] [-F <file>]
        [--into-name <branch>] [<commit>…]
git merge (--continue | --abort | --quit)
```

DESCRIPTION

Incorporates changes from the named commits (since the time their histories diverged from the current branch) into the current branch. This command is used by `git pull` to incorporate changes from another repository and can be used by hand to merge changes from one branch into another.

Assume the following history exists and the current branch is `master`:

```
          A---B---C topic
         /
    D---E---F---G master
```

4. 브랜치 병합

개요

만들어진 각 브랜치에서 작업을 하다가 어느 시점에는 브랜치 작업을 마무리하고 기존 main 브랜치와 합해야 된다. 이것을 브랜치 병합(merge)이라고 부른다.

병합 형태

1) Fast-forward (빨리 감기 병합)
2) 병합 커밋 (병합시 commit 작업 필요)
3) 병합충돌

특징

커밋이 아닌 브랜치를 병합하는 것이다.

항상 현재 HEAD 브랜치에 병합한다. 따라서 기준 브랜치(main)로 이동해서 병합해야 된다.

5. Fast-forward (빨리감기 병합)

개요

만약 main 브랜치에서 분기한 후에 main 브랜치에서 변경 사항이 없다면 분기한 브랜치를 병합하는 것은 매우 간단하다.

분기한 브랜치에서 만든 최신 커밋을 main 브랜치가 참조하도록 하면 된다.

이것을 Fast-forward (빨리 감기 병합)라고 부른다.

가. 기준 브랜치인 main 으로 변경

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch
  hotfix
* main
```

나. main 브랜치와 hotfix 브랜치 병합

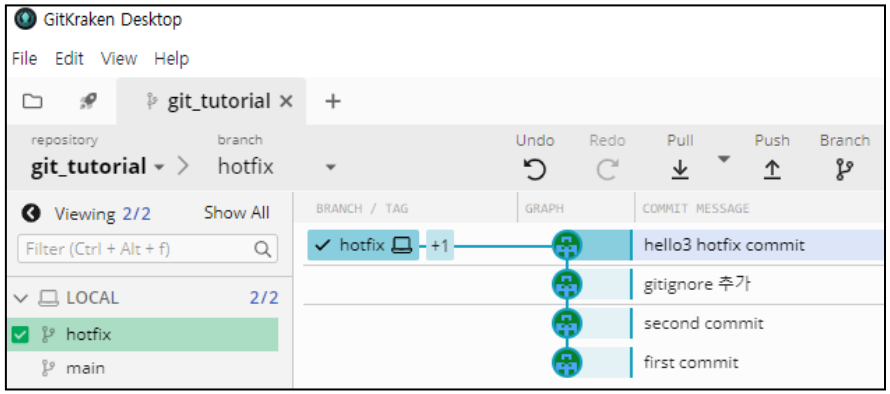
```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git merge hotfix
Updating 7506f4d..8e09b2f
Fast-forward      빨리 감기 병합
 hello.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

5. Fast-forward (빨리감기 병합)

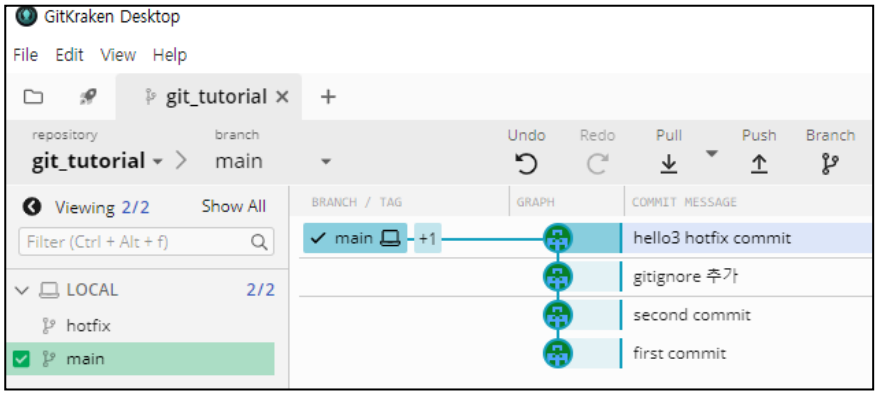
다. main 커밋 이력 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git log --oneline
8e09b2f (HEAD -> main, hotfix) hello3 hotfix commit
7506f4d gitignore 추가
2f0140b second commit
5b193a1 first commit
```

hotfix 브랜치 Graph



main 브랜치 Graph



6. 병합 커밋

개요

main 브랜치에서 분기한 후에 main 브랜치에서도 변경 사항(world.txt 추가)이 발생되고 분기한 hotfix 브랜치에서도 변경사항(hello.txt 수정)이 발생되었을 때 병합하는 방법이다. 이것을 병합 커밋 (merge commit)라고 부른다.

가. main 브랜치에서 world.txt 추가하고 커밋하기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ touch world.txt

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git add world.txt

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git commit -m  "world.txt 추 가 "
[main 9c6486f] world.txt 추 가
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 world.txt
```

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ ls -al
total 34
drwxr-xr-x 1 inky4 197609  0 Sep 20 16:04 ./
drwxr-xr-x 1 inky4 197609  0 Sep 20 12:15 ../
drwxr-xr-x 1 inky4 197609  0 Sep 20 16:04 .git/
-rw-r--r-- 1 inky4 197609 15 Sep 20 15:49 .gitignore
-rw-r--r-- 1 inky4 197609  0 Sep 20 15:49 Test.java
-rw-r--r-- 1 inky4 197609 28 Sep 20 16:04 hello.txt
drwxr-xr-x 1 inky4 197609  0 Sep 19 12:36 tmp/
-rw-r--r-- 1 inky4 197609  0 Sep 20 16:04 world.txt
```

나. hotfix 브랜치에서 hello.txt 수정하고 커밋하기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git switch hotfix
Switched to branch 'hotfix'

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ cat hello.txt
hello
hello2
hello3 hotfix
hello4 hotfix 수정        수정하기
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git commit -am "hello4 hotfix 수정"
[hotfix 31fc1b0] hello4 hotfix 수정
 1 file changed, 2 insertions(+), 1 deletion(-)
```

다. hotfix 브랜치에서 파일 목록 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ ls -al      world.txt 없음
total 34
drwxr-xr-x 1 inky4 197609  0 Sep 20 15:52 ./
drwxr-xr-x 1 inky4 197609  0 Sep 20 12:15 ../
drwxr-xr-x 1 inky4 197609  0 Sep 20 15:55 .git/
-rw-r--r-- 1 inky4 197609 15 Sep 20 15:49 .gitignore
-rw-r--r-- 1 inky4 197609  0 Sep 20 15:49 Test.java
-rw-r--r-- 1 inky4 197609 50 Sep 20 15:54 hello.txt
drwxr-xr-x 1 inky4 197609  0 Sep 19 12:36 tmp/
```

6. 병합 커밋

라. main 브랜치에서 hello.txt 파일 내용 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ cat hello.txt
hello
hello2
hello3 hotfix
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
```

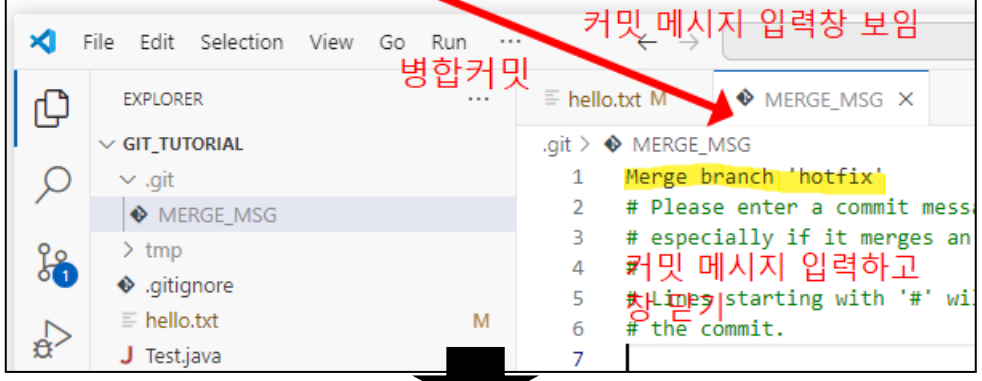
hello4 hotfix 수정 내용 없음

마. main 브랜치와 hotfix 브랜치 병합

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git merge hotfix
hint: Waiting for your editor to close the file...
```

병합커밋

커밋 메시지 입력창 보임



커밋 메시지 입력하고 창 닫기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git merge hotfix
Merge made by the 'ort' strategy.
 hello.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

Fast-forward 가 아닌 병합커밋으로 병합됨

바. main 브랜치에서 hello.txt 내용 확인하기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ cat hello.txt
hello
hello2
hello3 hotfix
hello4 hotfix 수정
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
```

hotfix 브랜치 내용과 병합됨

7. 병합 충돌 [conflict]

개요

git에서는 줄 단위(line)로 변경 여부를 확인한다.
따라서 서로 다른 브랜치에 같은 파일의 이름을 가지고 있으면서 같은 줄(line)을 수정하고 병합하면 병합 충돌(merge conflict)이 발생된다.
해결방법은 명시적으로 사용자가 충돌 위치를 수정하고 커밋해서 처리한다.

가. 현재 상태 보기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git status
On branch main
nothing to commit, working tree clean
```

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git status
On branch hotfix
nothing to commit, working tree clean
```

나. main 브랜치에서 hello.txt 수정

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ cat hello.txt
hello
hello2
hello3 hotfix
hello4 hotfix 수정
hello5 main
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git commit -am "hello5 main commit"
[main fa722a9] hello5 main commit
 1 file changed, 2 insertions(+), 1 deletion(-)
```

5번째줄에 hello5 main 추가하기

다. hotfix 브랜치에서 hello.txt 수정

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ cat hello.txt
hello
hello2
hello3 hotfix
hello4 hotfix 수정
hello5 hotfix 수정
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git commit -am "hello5 hotfix commit"
[hotfix 596992e] hello5 hotfix commit
 1 file changed, 2 insertions(+), 1 deletion(-)
```

5번째줄에 hello5 hotfix 추가하기

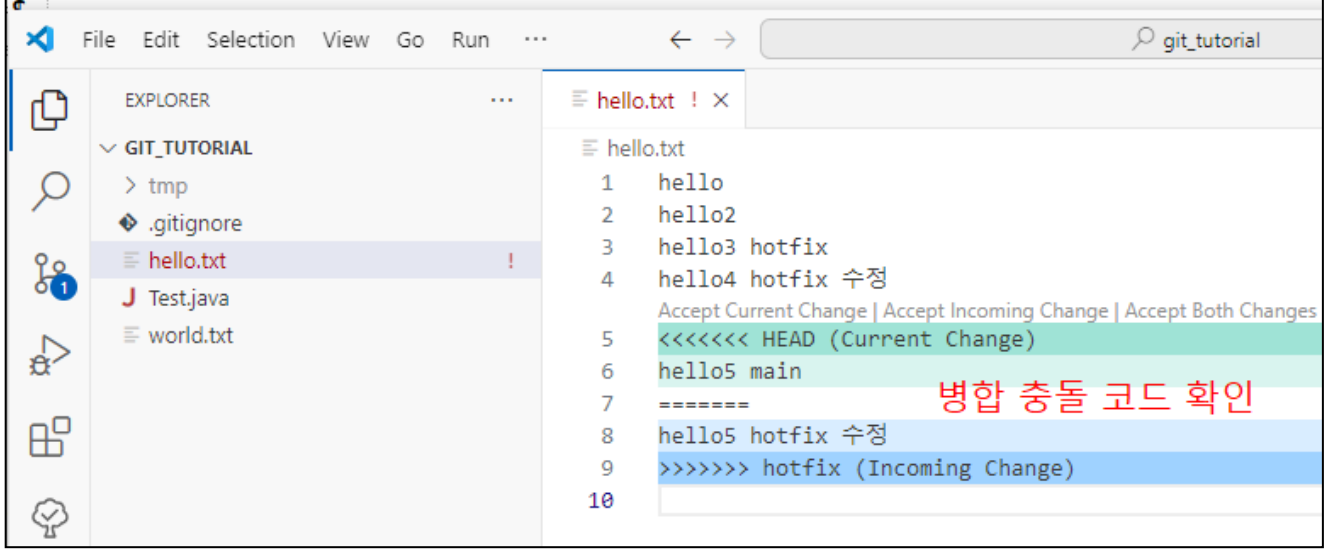
7. 병합 충돌 (conflict)

라. main 브랜치와 hotfix 브랜치 병합

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (hotfix)
$ git switch main
Switched to branch 'main'

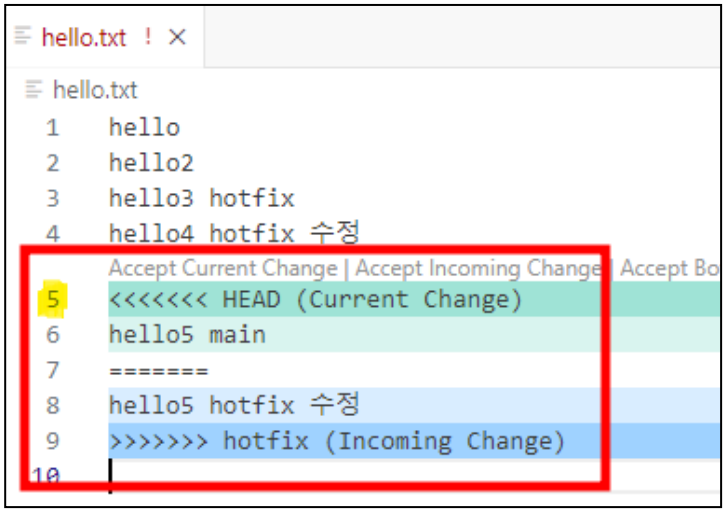
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git merge hotfix
Auto-merging hello.txt
CONFLICT (content): Merge conflict in hello.txt
Automatic merge failed; fix conflicts and then commit the result.

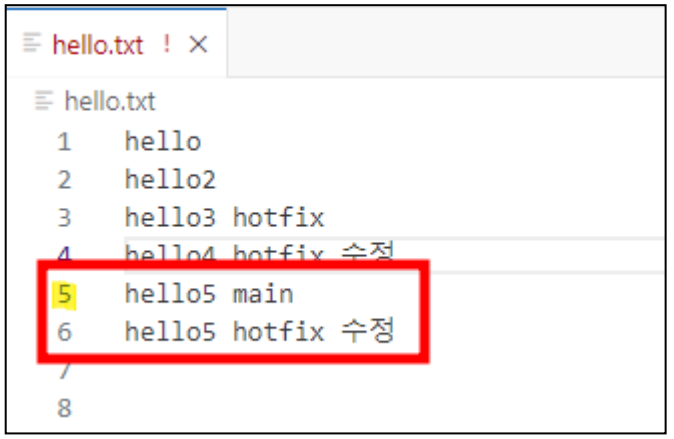
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main|MERGING)
```



7. 병합 충돌 [conflict]

마. 충돌시 명시적으로 해결





바. 변경사항 커밋하기

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main|MERGING)
$ git commit -am "main hotfix merge"
[main 034fb07] main hotfix merge

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git log --oneline
034fb07 (HEAD -> main) main hotfix merge
596992e (hotfix) hello5 hotfix commit
fa722a9 hello5 main commit
4a5f777 Merge branch 'hotfix'
31fc1b0 hello4 hotfix 수정
```

8. 병합시킨 브랜치 삭제

병합시킨 hotfix 브랜치 삭제

```
inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch
  hotfix
* main

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch -D hotfix
Deleted branch hotfix (was 596992e).

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$ git branch
* main

inky4@DESKTOP-KERHMJL MINGW64 ~/git_tutorial (main)
$
```

Thank you